

PROTEGE+

Canal Nacional de Denúncias para Proteção do Cidadão

HackGov · Atividade Visão de Futuro

Segurança da Informação · Redes de Comunicação · Estatística para Decisão · Blockchain

Tríade CIA	Redes	Correlação e Regressão	Smart Contract

Gabriel Vasconcellos Gomes

RM 561601

Turma 2SIOA · Barueri, São Paulo · Entrega individual

Vídeo de demonstração (YouTube, não listado): **youtu.be/XXXXXXXX (inserir após o upload)**
Repositório: **github.com/Gabriell230G/ProtegeMais-Fase-7**
Site publicado: **gabriell230g.github.io/ProtegeMais-Fase-7**

FIAP · 2026

▸ Ficha técnica

Item	Conteúdo
Projeto	Protege+, Canal Nacional de Denúncias
Entrega	Atividade Visão de Futuro (HackGov), sobre a base da Fase 6
Formato	Documento (este), apresentação de 10 slides, vídeo de até 5 minutos e código-fonte
Aluno	Gabriel Vasconcellos Gomes, RM 561601, turma 2SIOA
Vídeo, não listado	youtu.be/XXXXXXXX (inserir após o upload)
Repositório	github.com/Gabriell230G/ProtegeMais-Fase-7
Site publicado	gabriell230g.github.io/ProtegeMais-Fase-7
Back-end	Java 17, Spring Boot 3.3.4, Spring Security 6.3
Banco	H2 em arquivo (demonstração); PostgreSQL (produção); modelo físico em Oracle 19c
Contrato inteligente	Solidity 0.8.28, OpenZeppelin 5.1, Hardhat 2.29
Borda de rede	nginx 1.24 com TLS 1.2/1.3 e TLS mútuo para aparelhos
Testes automatizados	217 testes Java (JUnit 5) e 21 testes do contrato, sem falhas
Verificações executadas contra o sistema real	11 ataques simulados, 3 exercícios de recuperação, 39 verificações de rede
Classes Java de produção	93 (79 na Fase 6)
Endpoints REST	44 (38 na Fase 6)

▸ Sobre a estrutura deste documento

Este documento foi organizado para ser conferido item a item contra o enunciado da atividade. Cada uma das quatro Partes abre com um quadro **O que a atividade pede** e fecha com uma síntese que aponta onde cada pedido foi atendido, no texto e no repositório.

Os quatro princípios das Fases 5 e 6 continuam valendo, e um quinto foi acrescentado nesta entrega.

Toda decisão é justificada, e a alternativa descartada é nomeada. Uma escolha só é escolha quando havia outra opção viável. A Parte 4, por exemplo, compara a blockchain com quatro alternativas antes de adotá-la.

Toda afirmação técnica aponta para um arquivo. Quando o texto diz que uma defesa existe, ele nomeia a classe, o script ou a configuração em que ela está escrita.

Todo número é reproduzível. A base de demonstração é gerada com semente fixa (561601), e cada resultado citado tem o comando que o produz e o arquivo em que foi gravado.

Os limites são declarados. Cada Parte termina com o que não foi resolvido. Um documento que só relata acertos descreve uma demonstração, e não um sistema.

Novo nesta entrega: a defesa só conta depois de atacada. Nenhum controle de segurança deste documento é apenas descrito. Cada um foi exercitado contra o sistema rodando, por um roteiro que está no repositório e cujo resultado ficou gravado em `seguranca/resultados/`, `infra/resultados/` ou `blockchain/resultados/`. Esse critério encontrou defeitos que a leitura do código não tinha encontrado, descritos nas seções em que apareceram (2.5.4 e 3.8.2).

▸ Sumário

1. Introdução 5
- 1.1 O problema público e onde o projeto está 5
- 1.2 O que esta entrega acrescenta 5
- 1.3 Mapa: o que a atividade pede e onde está 6
- 1.4 Método de trabalho e critério de pronto 7

PARTE 1: Segurança da Informação 8
- 2.1 Ponto de partida e evolução medida 8
- 2.2 Evoluções desde a última entrega 9
- 2.3 A tríade CIA aplicada ao Protege+ 11
- 2.4 Políticas técnicas 12
- 2.5 Simulação de ataque e resiliência 14
- 2.6 Síntese da Parte 1 17

PARTE 2: Redes de Comunicação 18
- 3.1 Como o Protege+ usa a rede hoje 18
- 3.2 O modelo em camadas aplicado 18
- 3.3 A arquitetura de rede proposta 19
- 3.4 Segurança de rede e a tríade CIA 20
- 3.5 Resiliência 21
- 3.6 Confiança 21
- 3.7 Os aparelhos de campo na rede 22
- 3.8 O que foi implementado e testado 22
- 3.9 Limites declarados 23
- 3.10 Síntese da Parte 2 23

PARTE 3: Estatística para Decisão 24
- 4.1 De onde se partiu 24
- 4.2 As variáveis 24
- 4.3 Método 25
- 4.4 Resultados 25
- 4.5 Do número à decisão 28
- 4.6 Onde está no sistema 30
- 4.7 Cuidados na leitura 30
- 4.8 Síntese da Parte 3 30

PARTE 4: Blockchain e Smart Contract 31
- 5.1 O problema que a blockchain resolve aqui 31
- 5.2 Por que blockchain, e não outra coisa 31
- 5.3 Que blockchain 32
- 5.4 O contrato AncoraProtege 32
- 5.5 Testes do contrato 35
- 5.6 Blockchain e LGPD 35
- 5.7 Demonstração e exercício C 35
- 5.8 Custos medidos 36
- 5.9 Limites declarados 36
- 5.10 Síntese da Parte 4 37

6. Conclusão 38
- 6.1 O que foi entregue 38
- 6.2 O que não foi feito 38
- 6.3 Próximos passos, por prioridade 38
- 6.4 Observação de método 39

Referências 40

Anexo A. Rotas novas da API 41

Anexo B. Como reproduzir cada resultado 41

Anexo C. Arquivos novos desta entrega 42

Anexo D. Telas do sistema 43

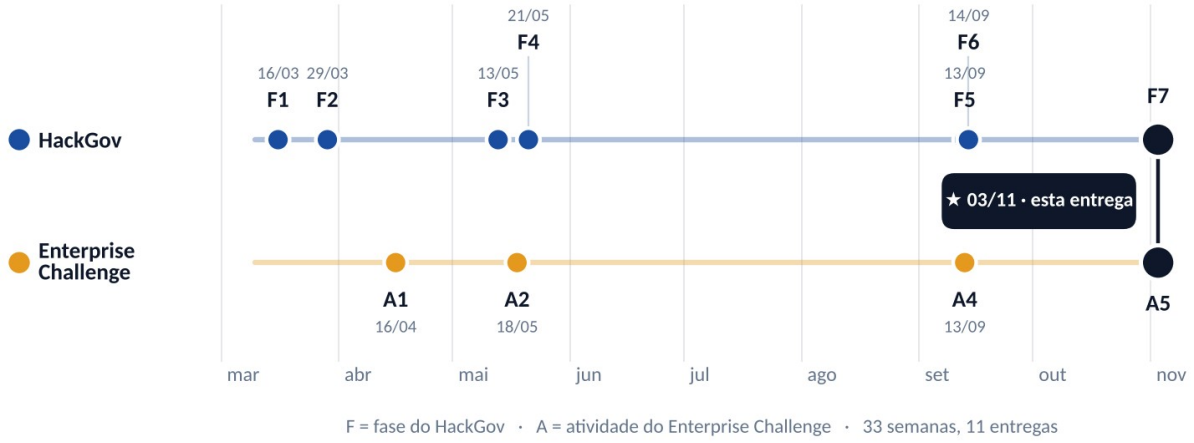
✦ 1. Introdução

■ 1.1 O problema público e onde o projeto está

O Protege+ é um canal de denúncia de violência doméstica, assédio, abuso e discriminação, pensado para o poder público. A cidadã denuncia pelo site, com ou sem identificação, recebe um protocolo para acompanhar o caso e pode sair da página com um atalho de emergência. O servidor público recebe a denúncia já triada por urgência, trabalha o caso num fluxo de status e deixa cada ação registrada numa trilha de auditoria.

O tamanho do problema justifica o cuidado. Em 2025, o Ligue 180 fez cerca de 1,09 milhão de atendimentos, e o Anuário Brasileiro de Segurança Pública 2026 registrou 1.571 feminicídios e 132.025 descumprimentos de medida protetiva, cerca de 362 por dia. Num sistema assim, um vazamento não é um incidente de TI: é a localização de uma vítima nas mãos de quem a ameaça.

Em seis fases, o projeto saiu de um site que guardava as denúncias no próprio navegador e chegou a um servidor Java com controle de acesso por perfil, trilha de auditoria encadeada por hash, triagem por IA, dispositivos de campo simulados e análise de focos de reincidência. A Fase 6 terminou com 79 classes, 38 endpoints e 136 testes.



▲ **Figura 1.** *As entregas do projeto em 2026. HackGov e Enterprise Challenge convergem nesta entrega, em 03/11.*

■ 1.2 O que esta entrega acrescenta

A atividade pede que o projeto olhe para o futuro em quatro direções: segurança da informação, redes de comunicação, estatística para decisão e blockchain. A resposta foi tratar as quatro como uma única pergunta: **o Protege+ merece a confiança de quem denuncia e de quem fiscaliza?**

- **Parte 1, segurança.** Uma revisão completa encontrou e corrigiu 26 falhas, várias graves, como a senha de administrador gravada no JavaScript público. As políticas de autenticação, criptografia, backup e acesso foram escritas a partir do código. Depois, 11 ataques e 3 exercícios de recuperação foram executados contra a API real.
- **Parte 2, redes.** O documento descreve a topologia de produção. Uma borda de rede real (nginx) foi configurada com HTTPS, TLS mútuo para os aparelhos e limites contra abuso, e testada com 39 verificações.
- **Parte 3, estatística.** O relatório descritivo da Fase 5 ganhou correlação e regressão, com p-valor e intervalo de confiança, calculadas no servidor e transformadas em decisões no painel.
- **Parte 4, blockchain.** Um contrato em Solidity ancora a trilha de auditoria e as evidências fora do servidor. Ele resolve um limite que o exercício de recuperação da Parte 1 revelou.



▲ **Figura 2.** *A entrega em seis números. Cada um foi medido contra o sistema rodando e está gravado no repositório.*

As quatro Partes se encadeiam. O exercício de recuperação da Parte 1 mostrou que restaurar um backup corta o fim da trilha sem deixar rastro, e esse achado motivou a Parte 4. A Parte 2 descobriu que, atrás de um proxy, a trilha gravaria o endereço errado, o que levou a uma correção no código da Parte 1. A Parte 3 mostrou que o score de confiabilidade não prevê nada, e isso virou recomendação de produto.

■ 1.3 Mapa: o que a atividade pede e onde está

▼ **Tabela 1.** Cada pedido da atividade, a seção que o responde e o artefato que o comprova.

Pedido	Seção	Artefato no repositório
Parte 1: evoluções desde a última entrega	2.1 e 2.2	`git log`, commits da Fase 7
Parte 1: tríade CIA	2.3	Tabela de controles por pilar
Parte 1: políticas de autenticação, criptografia, backup e controle de acesso	2.4	`SecurityConfig`, `GuardiaoDeSegredos`, `BackupService`, `FluxoStatus`
Parte 1: simular um ataque e descrever a resiliência	2.5	`seguranca/simular_ataques.py`, `seguranca/resultados/`
Parte 2: o que de redes se aplica ao projeto	3.1 a 3.3	Topologia de referência (figura da seção 3.3)
Parte 2: segurança, resiliência e confiança em redes, ligadas à CIA	3.4 a 3.7	`infra/nginx/protege.conf`, `infra/pki/`
Parte 2: comprovação	3.8	`infra/testar-rede.sh`, `infra/resultados/`
Parte 3: correlação e regressão no relatório	4.1 a 4.7	`Inferencia.java`, `AnalisePreditivaService`, aba Estatística
Parte 3: consultas SQL equivalentes	4.7	`database/07_estatistica_fase7.sql`
Parte 4: blockchain aplicada ao projeto	5.1 a 5.3	Comparação com alternativas
Parte 4: protótipo de smart contract em Solidity	5.4 a 5.8	`blockchain/contracts/AncoraProtege.sol`, testes e roteiros

■ 1.4 Método de trabalho e critério de pronto

O critério de pronto da Fase 6 tinha três degraus: compila e passa nos testes, a aplicação sobe, e a funcionalidade é exercida pela interface. Esta entrega acrescentou um quarto degrau para tudo o que é segurança: **a defesa é atacada e o resultado fica gravado**. Por isso há três pastas novas de evidência no repositório.

Pasta	O que contém	Resultado
`seguranca/`	Roteiro com 11 ataques contra a API e dois exercícios com o servidor parado (adulteração direta do banco e perda total com restauração)	11 bloqueados, 0 falhas; adulteração detectada; banco restaurado
`infra/`	Borda de rede nginx, autoridade certificadora de dispositivos e roteiro de verificação	39 verificações, 0 falhas; painel inteiro aberto por HTTPS
`blockchain/`	Contrato, 21 testes, roteiros de ancoragem e verificação, exercício de restauração contra a blockchain	21 testes passando; corte da trilha detectado

Os mesmos roteiros rodam de novo a cada push, numa máquina limpa do GitHub Actions (`.github/workflows/provas.yml`): testes Java, testes do contrato, os 11 ataques e as 39 verificações de rede. Uma prova que só rodou uma vez, na máquina de quem a escreveu, vale menos do que uma que qualquer pessoa vê rodar.

Todo o trabalho desta fase foi feito no repositório `ProtegeMais-Fase-7`, em commits separados por Parte, para que a evolução possa ser conferida commit a commit. As telas do sistema rodando estão no Anexo D.

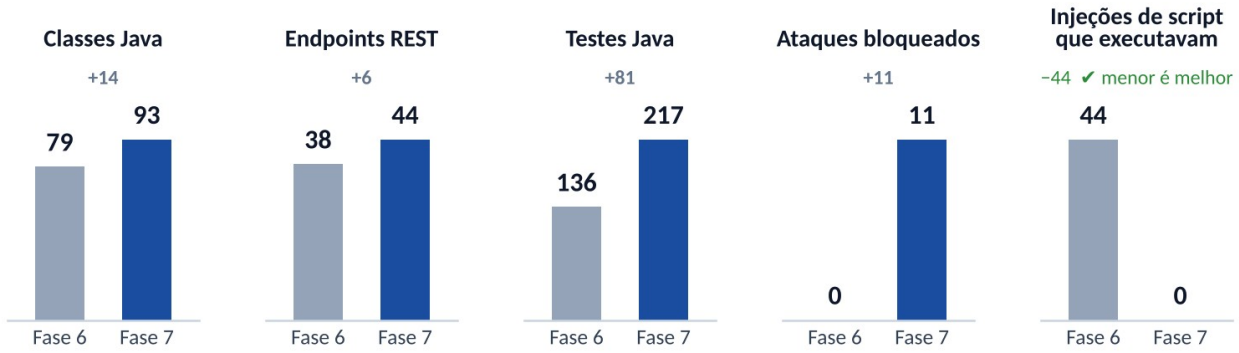
Fontes dos números da seção 1.1: Ministério das Mulheres (Ligue 180, balanço 2025) e Fórum Brasileiro de Segurança Pública, Anuário 2026, via CartaCapital. Referências completas no final.

☂ PARTE 1: Segurança da Informação

> ✎ **O QUE A ATIVIDADE PEDE**
>
> Relatar as evoluções do projeto desde a última entrega.
>
> Aplicar a tríade da segurança da informação (confidencialidade, integridade e disponibilidade) ao projeto.
>
> Definir as políticas técnicas de autenticação, criptografia, backups e controle de acesso.
>
> Simular um cenário de ataque e descrever como o sistema responde e se mantém resiliente.

■ 2.1 Ponto de partida e evolução medida

A Fase 6 entregou um sistema que funcionava e cujo discurso de segurança era bom: privacidade por desenho, menor privilégio, trilha imutável. A primeira tarefa desta fase foi conferir se o código sustentava o discurso. Em parte, não sustentava. A tabela abaixo mostra a evolução número a número; a seção 2.2 mostra o que foi corrigido.



▲ **Figura 3.** *Fase 6 e Fase 7 lado a lado. Cada painel tem a própria escala; no último, menos é melhor.*

▼ **Tabela 2.** Indicadores da Fase 6 e desta entrega, medidos no repositório.

Indicador	Fase 6	Fase 7	De onde vem a diferença
Classes Java de produção	79	93	Segurança (guardião de segredos, filtros, máquina de estados), backup, estatística, ancoragem
Endpoints REST	38	44	Backup (2), troca de perfil da demonstração, regressão, ancoragem (2)
Testes Java	136	217	81 novos, quase todos de segurança e estatística
Testes do contrato inteligente	-	21	Pasta `blockchain/`
Ataques executados contra a API	0	11	`seguranca/simular_ataques.py`
Verificações de rede	0	39	`infra/testar-rede.sh`
Pontos de injeção de script (XSS) que executavam	44	0	Teste no navegador antes e depois
Scripts SQL	5	7	Restrições da Fase 7 e consultas estatísticas
Backup	não havia	diário, verificado por hash	`BackupService` e `restaurar-backup.ps1`

■ 2.2 Evoluções desde a última entrega

As evoluções se dividem em cinco frentes. A primeira é a maior e a menos visível: uma revisão de segurança feita como um atacante faria, lendo o JavaScript público, o código do servidor e a configuração. As outras quatro são as Partes deste documento.

▸ 2.2.1 Revisão de segurança: 26 falhas encontradas e corrigidas

A revisão foi feita em quatro passadas independentes (API, front-end, banco e scripts, compilação e testes), cada uma por um revisor que não tinha visto as correções da anterior. Cada falha recebeu uma correção e, sempre que possível, um teste que falharia se ela voltasse.

▼ **Tabela 3.** Falhas encontradas na base da Fase 6, por gravidade. C, I e D indicam o pilar da tríade atingido.

#	Como estava na Fase 6	Risco	Correção na Fase 7	CIA
1	Senha do ADMIN escrita no `backend.js` público, com login automático	Qualquer visitante operava como administrador	Sem senha no cliente; troca de perfil da demonstração pedida ao servidor, só por ADMIN, e auditada	C I
2	Segredo do JWT com valor padrão publicado no repositório	Forjar token de qualquer perfil	`GuardiaoDeSegredos`: segredo sorteado a cada execução na demonstração; em produção a API não sobe com segredo fraco	C I
3	Chave dos aparelhos IoT fixa (`protege-iot-2026`)	Disparar alarmes falsos de pânico	Chave sorteada ou obrigatória; conferida antes de ler o corpo	I D
4	Console do banco H2 aberto e proteção contra embutir a página desligada	Acesso direto ao banco	Console desligado por padrão e só local; `frameOptions sameOrigin`	C I
5	44 pontos do front-end inseriam dados sem escape (incluindo `?q=` refletido na busca)	Script de terceiro roda no painel do gestor	Funções `escHtml`, `escAttr`, `jsArg`, `dataUrlSegura` em todos os módulos	C I
6	Protocolo sequencial (`#2026-0045`)	Enumerar casos pela consulta pública	Protocolo `#ANO-XXXX-XXXX-XXXX` com `SecureRandom`, cerca de 59 bits	C
7	Limite de requisições pelo primeiro item do `X-Forwarded-For`	Zerar o limite forjando o cabeçalho	Endereço real da conexão; cabeçalho só com proxy declarado confiável	D
8	AUDITOR lia relatos e evidências; ATENDENTE excluía, reatribuía e desfazia	Excesso de privilégio	Matriz de perfis por rota no `SecurityConfig`	C I
9	Perfil lido do token, válido até expirar	Usuário removido continua entrando por 8 h	Usuário e perfil conferidos no banco a cada requisição	C
10	Status mudava para qualquer valor (concluído voltava a novo)	Fraude no fluxo e nas métricas	Máquina de estados `FluxoStatus`	I
11	Relato editável depois do registro	Alterar a prova	Relato imutável (`EdicaoDoCaso`)	I
12	Fotos guardadas com metadados (EXIF, GPS)	A foto revela onde a vítima estava	Metadados removidos de JPEG e PNG antes do hash	C
13	Evidências de caso excluído continuavam no disco e acessíveis	Violação da LGPD	Arquivos apagados depois do commit da exclusão	C

#	Como estava na Fase 6	Risco	Correção na Fase 7	CIA
14	Relato integral enviado ao Gemini, com a chave na URL e sem tempo limite	Dado sensível fora do sistema; chave em log	Relato não sai por padrão (mascarado se ligado); chave no cabeçalho; 15 s de limite	C D
15	Urgência devolvida pela IA aceita sem validação	Modelo rebaixa caso crítico	Urgência final = a maior entre regras e modelo	I
16	Resposta da tornozeleira permitia inferir o perímetro	Agressor localiza a vítima	Resposta sem informação de perímetro	C
17	Login com tempo diferente para e-mail inexistente	Descobrir quem tem conta no órgão	BCrypt executado nos dois casos	C
18	Sem limite de tamanho de corpo	Esgotar memória	Filtro de 256 KB (413)	D
19	Expressões regulares com retrocesso exponencial no mascaramento	Travar a CPU com um texto	Expressões possessivas e limitadas; teto de 5.000 caracteres	D
20	Quebra de linha de campos do cliente ia para o log	Forjar linhas de log	`LogSeguro`	I
21	Não havia backup	Perda total do banco	Backup diário verificado por hash, com restauração	D I
22	Modo de demonstração ouvia em todas as interfaces de rede	Expor a demonstração na rede local	Só `127.0.0.1` em demonstração	C
23	Com 401 ou 403, o painel mostrava dados locais como se fossem do servidor	Decisão sobre dado falso	Aviso de acesso negado; dados locais só com a API fora do ar	I
24	Coordenada exata do caso para o ATENDENTE	Exposição desnecessária	Coordenada aproximada (cerca de 100 m)	C
25	Atalho de saída rápida disparava durante a digitação	Vítima perde a denúncia	SHIFT 3 vezes ignorado em campos de texto	D
26	Atrás de um proxy, a trilha gravaria o IP do proxy para todos (achado da Parte 2)	Trilha sem valor de prova	Mesmo critério de origem do limite de requisições	I

> ⚠ **POR QUE ISSO IMPORTA**
>
> As falhas 1 e 2 bastavam, sozinhas, para anular todo o controle de acesso. Nenhuma delas aparecia nos documentos anteriores, que descreviam as defesas como prontas. A lição foi incorporada ao método (seção 1.4): **uma defesa só é registrada como pronta depois de atacada.**

▸ 2.2.2 As outras quatro frentes

- **Backup e recuperação** (seção 2.4.3): cópia diária do banco com SHA-256 registrado na trilha e num manifesto separado, retenção de 7 cópias e roteiro de restauração que confere o hash antes de restaurar.
- **Simulação de ataques** (seção 2.5): 11 cenários automatizados contra a API real e 3 exercícios de recuperação.
- **Rede** (Parte 2), **estatística** (Parte 3) e **blockchain** (Parte 4).

■ 2.3 A tríade CIA aplicada ao Protege+

Num canal de denúncia, cada pilar da tríade tem um rosto. A confidencialidade protege a vida de quem denuncia; a integridade protege o valor de prova do que foi registrado; a disponibilidade protege o momento da emergência, que não espera o sistema voltar.

▼ **Tabela 4.** O que cada pilar significa no Protege+ e os controles que o sustentam.

Pilar	O que significa aqui	O que acontece se falhar	Controles implementados
Confidencialidade	Só quem precisa vê quem denunciou, o que foi relatado e onde a vítima está	O agressor descobre a denúncia ou o endereço; a vítima desiste de denunciar	Perfis por rota; anonimato sem nome nem contato no servidor; coordenada aproximada; EXIF removido; relato fora da IA; protocolo imprevisível; TLS; CORS; trilha de quem leu cada caso
Integridade	O registro é exatamente o que foi denunciado e o histórico do que o órgão fez	Prova contestada em juízo; fraude interna esconde um caso	Relato imutável; máquina de estados; trilha encadeada por SHA-256; hash de cada evidência; backup com manifesto; ancoragem em blockchain (Parte 4)
Disponibilidade	O canal recebe a denúncia quando ela acontece, inclusive sob ataque	A vítima não consegue pedir ajuda no momento crítico	Limites por origem e por tamanho; tempo limite na IA; backup e restauração; página de queda com 190 e 180; modo offline no navegador; borda de rede (Parte 2)



▲ **Figura 4.** *As 26 falhas por pilar da tríade, com os controles principais de cada um. Uma falha pode atingir mais de um pilar.*

Os pilares às vezes puxam em sentidos opostos, e o projeto precisou escolher.

- **Disponibilidade contra confidencialidade.** O modo offline deixa a cidadã registrar a denúncia sem rede, mas isso guarda dado no aparelho, que pode ser do agressor. A escolha foi guardar o mínimo, ofuscado, e oferecer a saída rápida que apaga tudo.
- **Integridade contra o direito de exclusão (LGPD, art. 18).** Uma trilha imutável parece contradizer a exclusão. A solução é a exclusão lógica com anonimização do conteúdo: a trilha registra que o caso 42 foi

excluído, mas nunca copia o relato. Na Parte 4, a mesma tensão reaparece na blockchain, resolvida com compromissos com sal.

- **Confidencialidade contra a triagem por IA.** Mandar o relato à IA melhora a triagem, mas tira o dado do sistema. A escolha foi não mandar por padrão: as regras locais já classificam, e o modelo só pode subir a urgência, nunca baixar.

■ 2.4 Políticas técnicas

As políticas abaixo descrevem o que o sistema faz hoje, com o arquivo em que cada regra está, e o que muda em produção. Elas foram escritas a partir do código, e não o contrário.

▸ 2.4.1 Autenticação

Regra	Como está implementada
Servidores entram com e-mail e senha; a senha é guardada só como hash BCrypt	`AuthController`, `SecurityConfig` (`BCryptPasswordEncoder`)
A sessão é um JWT assinado com HMAC-SHA256, válido por 8 horas	`JwtService`; `jwt.expiration=28800000`
O segredo do JWT nunca tem valor padrão. Na demonstração é sorteado a cada execução; em produção, a API se recusa a subir sem um segredo forte	`GuardiaoDeSegredos` (executa antes do Spring montar o contexto)
A cada requisição, usuário e perfil são relidos do banco: remover ou rebaixar alguém vale na hora	`JwtAuthFilter`
Login limitado a 20 tentativas por minuto por origem; e-mail inexistente e senha errada levam o mesmo tempo	`RateLimitFilter`; `AuthController` (hash falso)
Toda tentativa de login, aceita ou recusada, entra na trilha	`AuditoriaService.registrarLogin`
Em produção, as contas de demonstração são apagadas e a senha do ADMIN vem da variável `ADMIN_SENHA`	`SecurityConfig.seedUsuarios`
Aparelhos de campo se autenticam com chave própria (`X-Device-Key`) e, atrás da borda, também com certificado (TLS mútuo)	`ChaveDispositivoFilter`; `infra/nginx/protege.conf`
A cidadã não tem conta: acompanha o caso pelo protocolo, de 59 bits, com limite de 30 consultas por minuto	`DenunciaService.novoProtocolo`

Próximo passo: segundo fator para servidores. O caminho natural no setor público é o login gov.br (OpenID Connect) com conta nível prata ou ouro, que já traz segundo fator e dispensa o órgão de guardar senhas.

▸ 2.4.2 Criptografia

Onde	Algoritmo	Para quê	Arquivo
Em trânsito	TLS 1.2 e 1.3, troca de chaves ECDHE, cifras AES-GCM e ChaCha20	Sigilo e integridade entre navegador, aparelho e servidor	`infra/nginx/protege.conf`
Senhas	BCrypt (custo 10)	Hash lento, com sal, resistente a força bruta	`SecurityConfig`
Sessão	HMAC-SHA256 (JWT)	Token não pode ser alterado nem forjado	`JwtService`

Onde	Algoritmo	Para quê	Arquivo
Trilha de auditoria	SHA-256 encadeado	Detectar registro alterado, removido ou inserido	`AuditoriaService`
Evidências	SHA-256 do arquivo	Provar que o arquivo não mudou desde o envio	`EvidenciaService`
Backups	SHA-256 no manifesto e na trilha	Recusar restaurar uma cópia alterada	`BackupService`
Protocolo	`SecureRandom`	Número que não pode ser adivinhado	`DenunciaService`
Blockchain	SHA-256 com sal de 32 bytes, árvore Merkle (Keccak-256)	Ancorar sem expor dado	`blockchain/`

O que ainda não é cifrado, declarado. O arquivo do banco H2 e a pasta de evidências ficam em claro no disco do servidor; em produção, a política é usar volume cifrado e PostgreSQL com o diretório de dados cifrado. A "cifra" do armazenamento local do navegador (`js/lgpd.js`) é na verdade uma ofuscação por XOR com a chave no próprio cliente: impede a leitura casual, não uma perícia. Por isso a política para o navegador é outra: guardar o mínimo e apagar com a saída rápida. A troca por AES-GCM da Web Crypto, com chave não exportável, está nos próximos passos.

▸ 2.4.3 Backups

Regra	Valor
Frequência	Diária, às 02:30 (`BACKUP_CRON`); manual pelo ADMIN, com intervalo mínimo de 10 minutos
Método	Cópia online do H2 (`BACKUP TO`), sem parar a API
Retenção	7 cópias (`BACKUP_RETENCAO`)
Integridade	SHA-256 de cada cópia gravado em dois lugares: na trilha de auditoria e em `manifesto.sha256`, fora do banco
Restauração	`restaurar-backup.ps1`: recusa rodar com a API no ar, confere o hash contra o manifesto e devolve o banco original se a extração falhar
Destino em produção	Outro disco ou volume (`BACKUP_DIR`), mais uma cópia fora do local e cifrada: regra 3-2-1 (3 cópias, 2 mídias, 1 fora)
Teste	Exercício B da seção 2.5: banco apagado e restaurado em 10 segundos, com as 195 denúncias

Objetivos de recuperação. Com backup diário, a perda máxima de dados (RPO) é de 24 horas. Para um canal de denúncias isso é muito, e por isso a política de produção soma ao backup a réplica contínua do PostgreSQL (Parte 2), que reduz o RPO a segundos. O tempo de restauração (RTO) medido no exercício foi de 10 segundos para o banco de demonstração; em produção, a meta é de 1 hora, incluindo a decisão de restaurar.

▸ 2.4.4 Controle de acesso

O modelo é por perfil (RBAC), com dois princípios: **menor privilégio** (cada perfil tem só o que a função exige) e **segregação de funções** (quem opera não fiscaliza, e quem fiscaliza não opera). A regra está no servidor, em `SecurityConfig`; o site só esconde o que o servidor já recusaria.

▼ **Tabela 5.** Matriz de acesso por perfil. Sim significa permitido pelo servidor; não, recusado com 403.

Ação	ADMIN	GESTOR	ATENDENTE	AUDITOR
Ver indicadores agregados e estatística	sim	sim	sim	sim
Ler o relato e a evidência de um caso	sim	sim	sim	não
Ver a identificação de quem denunciou	sim	sim	não	não
Avançar o status do caso	sim	sim	sim	não
Voltar etapa, desfazer, reatribuir, excluir	sim	sim	não	não
Consultar a trilha de auditoria	sim	não	não	sim
Gerar backup	sim	não	não	não
Listar backups	sim	não	não	sim

Três regras completam a matriz. O status só anda pelos caminhos previstos (`FluxoStatus`). O relato não muda depois de registrado. E toda leitura de caso individual entra na trilha como consulta sensível, inclusive a do próprio auditor: quem fiscaliza também é fiscalizado.

■ 2.5 Simulação de ataque e resiliência

▸ 2.5.1 O cenário

O cenário foi escrito a partir de quem realmente ameaça um canal de denúncia: o agressor que suspeita ter sido denunciado. Ele não é um especialista, mas tem motivação, tempo, às vezes o celular da vítima e às vezes um conhecido dentro do órgão. Ele quer quatro coisas: descobrir se há denúncia, saber onde a vítima está, apagar o rastro e, se nada der certo, derrubar o canal.

Cada objetivo virou um ou mais ataques automatizados no roteiro `seguranca/simular_ataques.py`, que roda contra a API real e grava a resposta de cada defesa. Dois ataques exigem acesso ao servidor e foram feitos como exercícios, com a API parada, pelo roteiro `seguranca/exercicios_insider_e_recuperacao.sh`.

▸ 2.5.2 Os ataques e as respostas



▲ **Figura 5.** *Os 11 ataques: o objetivo do agressor, como foi tentado, a resposta da API e o pilar protegido.*

▼ **Tabela 6.** Execução registrada em 27/10/2026, 20:34. Resultado: 11 bloqueados, 0 falhas.

#	Objetivo do atacante	Ataque	Resposta do sistema	CIA
C01	Entrar como administrador	Token de ADMIN assinado com o segredo público da Fase 6; token sem assinatura; assinatura alterada	401, 401, 401	C I
C02	Ver mais do que o perfil permite	Atendente lê auditoria, exclui e reatribui; auditor lê relato e evidência; gestor gera backup	403 nos seis	C
C03	Descobrir a senha de um servidor	25 senhas erradas, cada uma com IP falso no `X-Forwarded-For`	16 × 401 e depois 429	C D
C04	Descobrir se há denúncia	40 protocolos sorteados na consulta pública	0 encontrados; 429 depois de 30	C
C05	Rodar script no painel do gestor	Denúncia anônima com `<img onerror>` e `<script>`	Guardado como texto; consulta pública sem o relato; painel escapa	I
C06	Enterrar casos reais	8 denúncias falsas seguidas	4 aceitas, 4 × 429	D
C07	Plantar malware como prova	Executável renomeado para `.jpg`; arquivo de 7 MB	415 e 413	I D
C08	Esgotar a memória	JSON de 400 KB sem login	413 antes de ler o corpo	D
C09	Localizar a vítima pela tornozeleira	Aparelho sem chave; chave antiga; sondar o perímetro	401, 401, resposta sem dado de localização	C
C10	Sequestrar os dados	Ransomware altera o arquivo de backup	Backup marcado como ALTERADO; íntegro de novo depois de restaurar	I D

#	Objetivo do atacante	Ataque	Resposta do sistema	CIA
C11	Não deixar rastro	Conferir a trilha depois de tudo	22 recusas registradas, cadeia SHA-256 intacta	I

▸ 2.5.3 Os exercícios com acesso ao servidor

Exercício A: um funcionário altera o banco direto. Com a API parada, ele abre o arquivo do banco, troca o usuário do registro 3 da trilha (para esconder um acesso) e apaga o registro 5. Ao religar, o auditor pede a verificação e recebe: *registro 3: o conteúdo foi alterado após a gravação*. A cadeia não impede a fraude de ser escrita, mas impede que ela passe despercebida, e o arquivo adulterado é guardado como prova.

Exercício B: perda total do banco. O banco é apagado, como num disco queimado ou num ransomware. Antes de restaurar, o hash do backup é conferido contra o manifesto. Em 10 segundos a API volta com as 195 denúncias e a cadeia íntegra.

```
seguranca/resultados/exercicios-20261027-203427.txt
[20:35:50] Banco removido da pasta de dados. Restaurando do backup protege-20261027-203428-
469.zip
[20:35:50] SHA-256 atual:      1a3770f0c748d64d5d015a55f859c04a13b01575206a3d36e65b28565c3a13cf
[20:35:50] SHA-256 registrado: 1a3770f0c748d64d5d015a55f859c04a13b01575206a3d36e65b28565c3a13cf
[20:35:50] Confere: backup integro, pode restaurar
[20:36:00] API restaurada | denuncias: 195 | integridade:
{"totalDeRegistros":12,"integra":true}
```

O que o exercício B revelou. Antes da perda, a trilha tinha 30 registros; depois da restauração, 12. Os 18 registros feitos depois do backup sumiram, e a verificação interna diz que a cadeia está intacta, porque o corte foi no fim. Esse é o limite de qualquer cadeia de hash guardada no mesmo lugar que ela protege: quem controla o servidor pode voltar no tempo. A Parte 4 resolve isso guardando a impressão digital da trilha fora do servidor, e o exercício C (seção 5.7) repete esta cena com a blockchain detectando o corte.

▸ 2.5.4 Como o sistema se mantém resiliente

A resiliência foi organizada pelas funções do NIST Cybersecurity Framework 2.0: o sistema precisa impedir o ataque, perceber o que passou, responder e voltar a funcionar.

Função	No Protege+	Evidência
Proteger	Perfis por rota, segredos fortes, escape de saída, limites, TLS, TLS mútuo, protocolo imprevisível	C01 a C09
Detectar	Trilha encadeada por hash; registro das recusas; verificação de integridade; hash de backups e evidências; ancoragem externa	C10, C11, exercícios A e C
Responder	Revogar a conta (vale na hora, pois o perfil é relido a cada requisição); revogar o ancorador comprometido; contestar uma ancoragem; guardar o arquivo adulterado como prova	Exercício A; testes do contrato
Recuperar	Backup verificado e restauração em segundos; réplica em produção; página de queda com 190 e 180 enquanto o serviço volta	Exercício B; seção 3.5

Ao rodar os ataques contra o sistema, apareceram também defeitos que não eram de segurança, mas que a revisão de código não tinha visto: a aba de auditoria abria vazia quando acessada pelo menu, o perfil auditor enchia a própria trilha com recusas porque o site pedia a lista de casos, e a edição de um caso exigia de novo o relato, que já era imutável. Os três foram corrigidos.

▸ 2.5.5 Limites declarados

- ⚠️ A simulação roda contra a instância de demonstração, na mesma máquina. Ataques de volume de rede (DDoS) não são simulados aqui; a defesa de volume é da Parte 2.
- ⚠️ O exercício A detecta a adulteração, mas não a impede. Impedir exige que o funcionário não tenha acesso ao arquivo do banco, o que é política de produção (acesso administrativo só por VPN e com registro).
- ⚠️ Não houve teste de engenharia social nem de segurança física. São os vetores mais prováveis contra um órgão público e dependem de treinamento, não de código.

■ 2.6 Síntese da Parte 1

Pedido	Onde foi atendido
✔️ Evoluções desde a última entrega	2.1 (indicadores) e 2.2 (26 falhas corrigidas e as outras quatro frentes)
✔️ Tríade CIA	2.3, com controles por pilar e as tensões entre pilares
✔️ Políticas de autenticação, criptografia, backup e controle de acesso	2.4.1 a 2.4.4, cada regra com o arquivo que a implementa
✔️ Simular um ataque e descrever a resiliência	2.5: 11 ataques e 3 exercícios executados, com resultados gravados em `seguranca/resultados/`

☷ PARTE 2: Redes de Comunicação

> ✎ **O QUE A ATIVIDADE PEDE**
>
> Identificar o que de redes de comunicação se aplica ao projeto.
>
> Tratar segurança, resiliência e confiança em redes, relacionando-as à tríade CIA.

■ 3.1 Como o Protege+ usa a rede hoje

Antes de propor arquitetura, foi preciso listar por onde os dados do Protege+ realmente passam. A lista surpreende: além da conversa óbvia entre navegador e servidor, há chamadas a quatro serviços de terceiros, e cada uma é uma decisão de privacidade.

▼ **Tabela 7.** Fluxos de rede do sistema, com o protocolo e a sensibilidade do que trafega.

Origem → destino	Protocolo	O que trafega	Sensibilidade
Navegador da cidadã → API	HTTPS (REST/JSON)	Denúncia, relato, evidências, consulta por protocolo	Máxima
Navegador do servidor público → API	HTTPS (REST/JSON), token JWT	Casos, identificação, trilha	Máxima
Aparelhos de campo → API	HTTPS, chave do aparelho e certificado	Acionamento de pânico, posição, relato do totem	Máxima (localização)
API → Google Gemini	HTTPS	Pergunta ao assistente; relato só se ligado, e mascarado	Alta, desligado por padrão
Navegador → IBGE	HTTPS	Lista de estados e cidades	Nenhuma
Navegador → Nominatim (OpenStreetMap)	HTTPS	Coordenada, quando a cidadã usa "minha localização"	Alta: revela onde a pessoa está a um terceiro
Navegador → CDNs (React, Chart.js, Leaflet)	HTTPS	Nenhum dado; só baixa bibliotecas	Baixa, mas a CDN vê o acesso
API → blockchain (Parte 4)	JSON-RPC sobre HTTPS	Só hashes e contadores	Nenhuma

Dois achados saíram dessa lista. O Nominatim recebe a coordenada de quem pediu "usar minha localização", o que só é aceitável com consentimento explícito e informado; em produção, o certo é um servidor de geocodificação próprio. E as bibliotecas vêm de CDNs públicas: uma CDN comprometida poderia servir um script alterado. A correção completa é hospedar as bibliotecas no próprio servidor ou usar verificação de integridade (atributo `integrity`, SRI); está nos próximos passos.

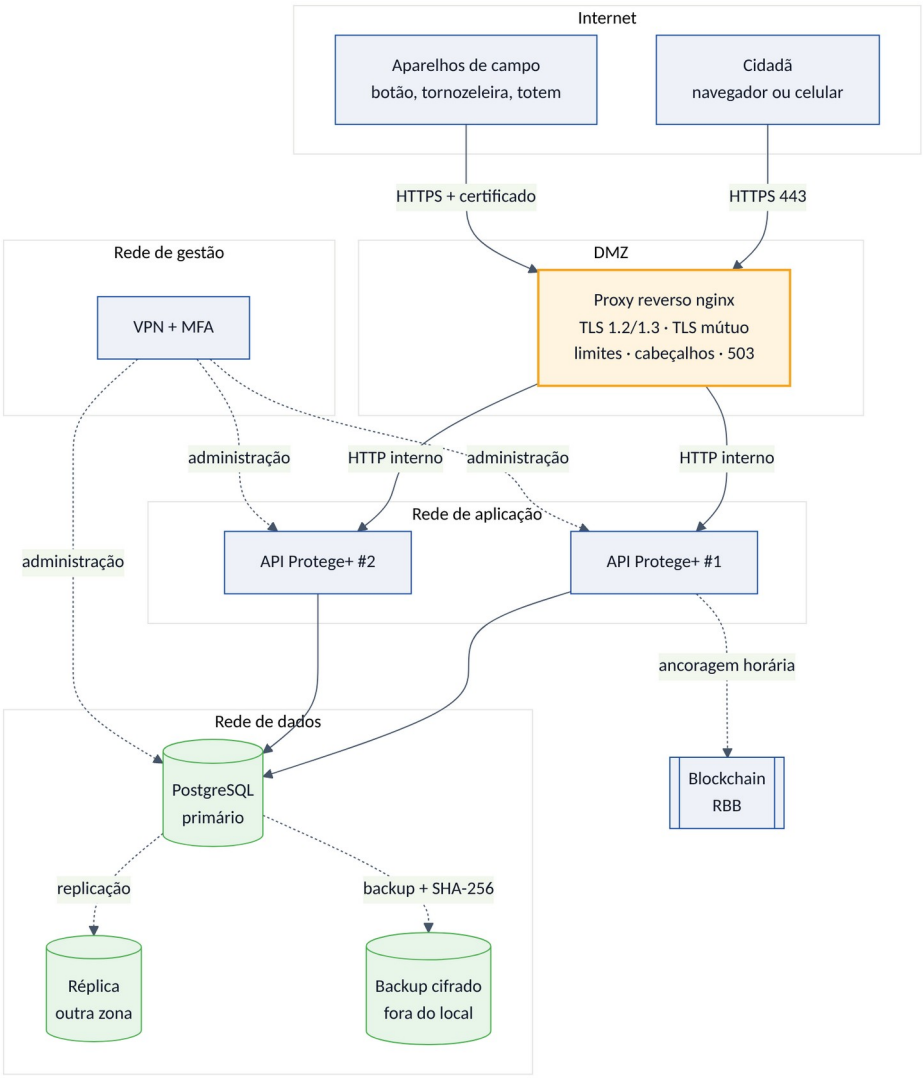
■ 3.2 O modelo em camadas aplicado

Cada camada do modelo TCP/IP tem um papel e uma defesa no Protege+. A defesa de uma camada não substitui a de outra: o TLS protege o transporte, mas não impede um script malicioso de chegar dentro de uma denúncia, que é problema da camada de aplicação.

Camada	No Protege+	Ameaça típica	Defesa
Aplicação	HTTP/2 e HTTP/1.1, REST com JSON; DNS	Injeção, abuso de rota, CSRF, XSS	Validação no servidor, perfis, CORS, CSP, escape de saída
Transporte (segurança)	TLS 1.2/1.3 sobre TCP 443	Interceptação, servidor falso	Certificado de AC confiável, HSTS, TLS mútuo para aparelhos
Transporte	TCP	Inundação de conexões, conexão lenta (slowloris)	Limite de conexões e tempos de espera no proxy
Rede	IP; sub-redes separadas por função	Varredura, acesso lateral	Segmentação (DMZ, aplicação, dados, gestão), firewall
Enlace e física	Datacenter ou nuvem; 4G dos aparelhos	Interceptação no rádio, roubo do aparelho	APN privada para os aparelhos; certificado revogável por aparelho

■ 3.3 A arquitetura de rede proposta

A figura abaixo mostra a topologia para operação real. A regra é que cada camada só converse com a vizinha: a internet só alcança o proxy, o proxy só alcança a API, e só a API alcança o banco. Quem administra entra por uma rede de gestão separada, por VPN com segundo fator.

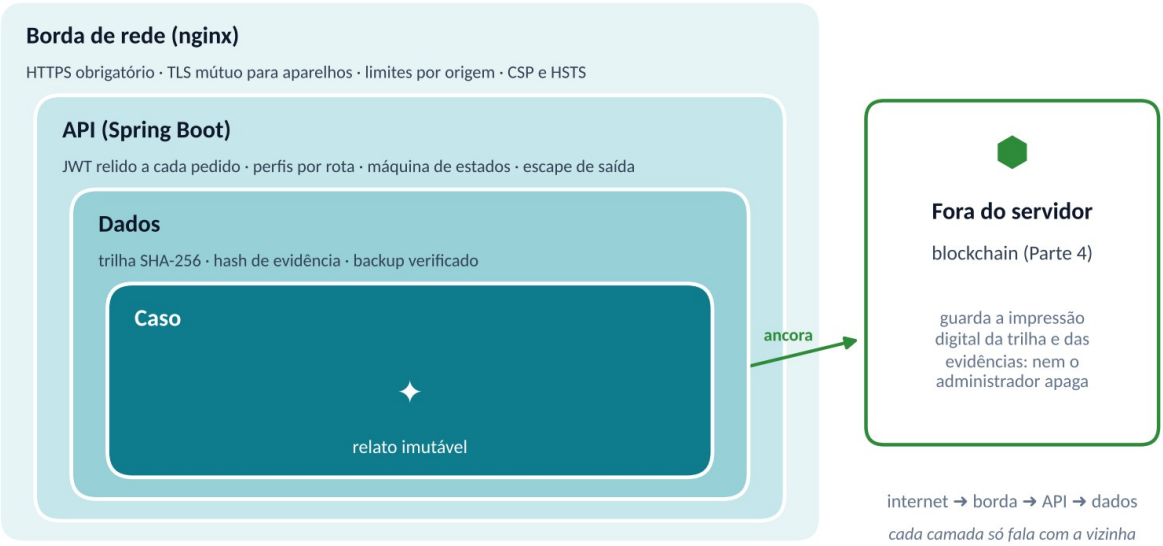


▲ **Figura 6.** *Topologia de referência. Em laranja, a borda que foi implementada e testada nesta fase.*

Zona	O que fica nela	O que pode entrar	Por quê
Internet	Cidadãs, servidores públicos, aparelhos de campo	-	-
DMZ	Proxy reverso (nginx)	Só portas 80 e 443 da internet	É a única peça exposta; se cair ou for invadida, não tem dado nenhum
Aplicação	Duas ou mais instâncias da API	Só o proxy, na porta interna	A API nunca recebe conexão direta da internet
Dados	PostgreSQL primário e réplica; backups	Só a API, na porta do banco	O dado mais sensível fica a dois saltos da internet
Gestão	Acesso administrativo	Só pela VPN, com segundo fator	O exercício A da Parte 1 mostrou o que um acesso ao banco permite

A rede de gestão aplica a ideia de confiança zero (NIST SP 800-207): estar dentro da rede do órgão não dá acesso a nada. Cada acesso é autenticado, autorizado e registrado, como já acontece com as requisições à API.

■ 3.4 Segurança de rede e a tríade CIA



▲ **Figura 7.** *Defesa em profundidade: cada camada tem a própria defesa, e a blockchain guarda a prova fora do servidor.*

▼ **Tabela 8.** Controles de rede por pilar. Os marcados com ✓ foram implementados e testados na seção 3.8.

Pilar	Controle de rede	Status
Confidencialidade	HTTPS obrigatório, TLS 1.2/1.3 com sigilo futuro (ECDHE); TLS 1.1 e anteriores recusados	✓
Confidencialidade	Só o front-end público é servido: código, banco, chaves, console do H2 e documentação da API ficam inacessíveis	✓
Confidencialidade	CORS: o navegador só deixa o endereço oficial usar a API	✓
Confidencialidade	Segmentação em zonas e VPN para administração	proposto
Integridade	TLS mútuo: cada aparelho prova quem é com certificado próprio	✓
Integridade	HSTS impede o rebaixamento para HTTP; CSP limita de onde vêm os scripts	✓
Integridade	O IP de origem gravado na trilha é o real, mesmo com cabeçalho forjado	✓

Pilar	Controle de rede	Status
Disponibilidade	Limites por origem na borda (denúncias, login, API) e por tamanho de corpo	✓
Disponibilidade	Conexões lentas de propósito são derrubadas em 10 segundos	✓
Disponibilidade	API fora do ar: resposta imediata em JSON com os telefones 190 e 180	✓
Disponibilidade	Serviço de mitigação de DDoS na frente do proxy; duas instâncias da API	proposto

■ 3.5 Resiliência

Num canal de emergência, resiliência não é só voltar a funcionar: é **continuar ajudando enquanto não funciona**. O projeto trabalha em três níveis.

- **Evitar a queda.** Duas instâncias da API atrás do proxy, que já tenta a outra quando uma falha (`proxy_next_upstream`). Banco com réplica contínua em outra zona. Dois provedores de internet no datacenter.
- **Absorver o ataque.** Limites por origem barram o abuso na borda, onde o custo é menor: o proxy é barato de replicar, a API não. Contra ataques de volume que saturam o link, a defesa precisa estar antes do datacenter, num serviço de mitigação (CDN com WAF), e isso é declarado como proposta.
- **Degradar com dignidade.** Se a API cai, a borda responde em milissegundos com uma mensagem em JSON que inclui os telefones 190 e 180, em vez de deixar a página carregando. Sem a API, o site continua registrando a denúncia no próprio aparelho e avisa que está desconectado; o reenvio automático quando a conexão voltar está nos próximos passos. E a saída rápida funciona mesmo sem rede.

Indicador	Meta de produção	Como se chega lá
Disponibilidade	99,9% ao mês (cerca de 43 min de parada)	Duas instâncias, réplica, monitoramento externo
RPO (dados perdidos)	Segundos	Réplica contínua do PostgreSQL; backup diário como segunda linha
RTO (tempo para voltar)	1 hora	Restauração verificada, testada em exercício (Parte 1)
Janela máxima de trilha sem testemunha	1 hora	Ancoragem horária em blockchain (Parte 4)

■ 3.6 Confiança

Confiança em rede é saber com quem se fala. O Protege+ precisa disso em três direções.

- **A cidadã confia que fala com o órgão.** O certificado do site vem de uma autoridade certificadora reconhecida (em produção, uma AC pública ou da ICP-Brasil), o HSTS impede que alguém a leve para uma versão sem cifra, e o endereço oficial do governo (`.gov.br`) é o que ela aprende a reconhecer.
- **O órgão confia que o aparelho é dele.** Cada botão de pânico, tornozeleira ou totem recebe um certificado próprio, emitido por uma autoridade certificadora de dispositivos separada da do site. Um aparelho roubado é revogado sozinho, sem trocar a chave de todos. É o que o teste do certificado "pirata" comprova: um certificado válido, com o nome de um aparelho real copiado, mas emitido por outra AC, é recusado.
- **Quem fiscaliza confia no registro.** A trilha encadeada e a ancoragem em blockchain (Parte 4) permitem que um auditor externo confira o histórico sem precisar confiar no servidor.

■ 3.7 Os aparelhos de campo na rede

Os aparelhos da Fase 6 falam com a API por HTTPS. Para operação real, a recomendação é que o modem 4G de cada aparelho use uma APN privada da operadora, que coloca o aparelho numa rede que não passa pela internet aberta. Sobre ela, o TLS mútuo continua valendo.

A alternativa avaliada foi o MQTT, protocolo próprio para IoT, com TLS na porta 8883. Ele seria melhor com milhares de aparelhos enviando posição a cada segundo, porque mantém a conexão aberta e tem o recurso *last will*, que avisa quando um aparelho para de responder. Com três tipos de aparelho e eventos raros, o HTTPS com certificado entrega a mesma segurança com uma peça a menos para operar. A decisão é revista quando a frota crescer.

■ 3.8 O que foi implementado e testado

A borda da topologia da seção 3.3 foi implementada de verdade, em `infra/nginx/protege.conf`, com a autoridade certificadora de dispositivos em `infra/pki/`. O roteiro `infra/testar-rede.sh` sobe o nginx na frente da API e confere cada defesa com requisições reais.

▸ 3.8.1 Resultado

▼ **Tabela 9.** Execução registrada em 27/10/2026, 21:31 (`infra/resultados/rede-20261027-213123.txt`). 39 verificações, 0 falhas.

Grupo	Verificações	Destaques
Transporte (TLS)	7	HTTP vira HTTPS; TLS 1.3 e 1.2 aceitos; TLS 1.1 recusado pelo servidor; troca de chaves ECDHE; cliente sem a AC recusa o servidor
Página e superfície exposta	16	HSTS, CSP, nosniff, X-Frame-Options e Referrer-Policy presentes; versão do servidor escondida; código, banco, chaves, simulador, console H2 e Swagger dão 404
Aparelhos de campo	5	Sem certificado: 403 no proxy; certificado pirata: 403; certificado certo com chave errada: 401 da API; os dois certos: 201
Rastreabilidade e CORS	4	Login forjando `X-Forwarded-For: 6.6.6.6` fica na trilha com o IP real; site de terceiros recebe 403
Disponibilidade	7	Rajada de denúncias barrada na 4a; JSON de 400 KB e arquivo de 7 MB barrados, evidência de 1 MB passa; API desligada responde 503 em 30 ms; conexão lenta derrubada em 10 s

```
infra/resultados/rede-20261027-213123.txt (trecho)
--- 3. Aparelhos de campo: certificado + chave
  [OK]     Sem certificado: barrado no proxy, nem chega a API
           obtido: 403 proxy
  [OK]     Certificado pirata (CN copiado de aparelho real, outra AC): barrado
           obtido: 403
  [OK]     Certificado valido com chave errada: a API recusa
           obtido: 401
  [OK]     Certificado valido e chave certa: aceito
           obtido: 201

RESULTADO: 39 verificacoes OK, 0 falhas
```

Além do roteiro, o painel completo foi aberto num navegador por HTTPS, através da borda, com login de administrador e passagem pelas 10 abas: nenhuma violação da política de conteúdo, nenhum erro de script e 15 rotas diferentes da API respondendo 200.

▸ 3.8.2 O que o teste encontrou

O roteiro falhou nas primeiras execuções, e cada falha era um defeito real que a leitura da configuração não tinha mostrado.

Defeito	Consequência	Correção
A rota `location ^~ /api/` impedia o nginx de avaliar as regras específicas, escritas como expressões regulares	Um aparelho sem certificado passava, e uma foto de 1 MB como evidência seria recusada	Prefixo comum, sem `^~`
Um `proxy_set_header` dentro da rota dos aparelhos apagava os cabeçalhos herdados, inclusive o `Host`	Aparelho legítimo recebia 400 do servidor de aplicação	Cabeçalho movido para o nível do servidor
A trilha de auditoria lia só o endereço da conexão	Atrás do proxy, todos os registros teriam o IP do proxy	Mesmo critério do limite de requisições, com teste unitário
A origem pública não estava na lista de CORS da API	Login pela borda recusado com 403	Variável `CORS_ORIGENS` documentada para produção; teste dos dois lados
A página chamava a API do IBGE, fora da política de conteúdo	Lista de cidades vazia	IBGE incluído em `connect-src`

■ 3.9 Limites declarados

- ⚠ A borda foi testada numa única máquina. Segmentação em zonas, VPN, réplica do banco e mitigação de DDoS estão descritas, mas não montadas.
- ⚠ A autoridade certificadora de dispositivos é um script com OpenSSL. Em produção, a chave da AC fica num equipamento dedicado (HSM), com lista de revogação publicada.
- ⚠ A política de conteúdo ainda precisa de `unsafe-inline`, porque o painel usa `onclick` no HTML. Trocar esses atributos por ouvintes de evento permitiria uma política estrita.
- ⚠ O roteiro de rede roda em Linux ou WSL; no Windows, só a geração de certificados funciona direto (Git Bash).

■ 3.10 Síntese da Parte 2

Pedido	Onde foi atendido
✔ O que de redes se aplica ao projeto	3.1 (fluxos reais), 3.2 (camadas), 3.3 (topologia por zonas), 3.7 (aparelhos)
✔ Segurança em redes ligada à CIA	3.4, com cada controle marcado como testado ou proposto
✔ Resiliência	3.5, com metas de disponibilidade, RPO e RTO
✔ Confiança	3.6: certificados do site, certificados por aparelho e verificação externa
✔ Comprovação	3.8: 39 verificações reais e 5 defeitos encontrados pelo teste

⚖️ PARTE 3: Estatística para Decisão

> ✎ **O QUE A ATIVIDADE PEDE**
>
> Aprimorar os relatórios estatísticos do projeto com correlação e regressão, de forma que apoiem decisões.

■ 4.1 De onde se partiu

Desde a Fase 5, a aba Estatística mostra um relatório descritivo calculado no servidor: média, mediana, desvio-padrão, quartis, valores atípicos e prazos por tipo e por urgência. Na base de demonstração, 196 denúncias, 123 concluídas, com média de 5,71 dias até a conclusão, mediana de 5,31 e 90% dos casos fechados em até 10,26 dias.

O relatório descritivo responde **como o tempo se distribui**. Ele não responde as perguntas que um gestor faz na segunda-feira de manhã: **o que faz um caso demorar? O que dá para mudar? Quantas pessoas a equipe precisa no mês que vem?** Correlação e regressão respondem exatamente isso, e foram construídas para responder com a força da evidência junto: nenhuma conclusão aparece sem p-valor e intervalo de confiança.

■ 4.2 As variáveis

A variável resposta é o **tempo até a conclusão**, em dias, de cada caso concluído. As explicativas são as que o sistema já registra no momento em que a denúncia chega, porque só elas servem para decidir antes de o caso atrasar.

Variável	Como é medida	Por que entra
Tempo até a conclusão (resposta)	Dias entre o registro e a conclusão	É o que o gestor quer reduzir
Gravidade do tipo	Peso do tipo: violência e abuso 5, assédio e discriminação 3, outros 2	A fila do VigIA prioriza por gravidade; a pergunta é se isso funciona
Urgência da triagem	Crítica 4, alta 3, média 2, baixa 1	Saber se a urgência atribuída muda o atendimento
Score de confiabilidade	0 a 100, calculado no registro	O score existe desde a Fase 2; nunca se mediu se ele prevê algo
Denúncia anônima	1 se anônima, 0 se identificada	Saber se o anonimato custa tempo para a vítima
Tamanho do relato	Número de caracteres	Relatos longos dariam mais trabalho?

Base de dados. A base é a de demonstração, gerada com semente fixa: 123 casos concluídos. Os números mostram o método funcionando, não a realidade de um órgão. Isso está escrito também na tela, num quadro de cuidados na leitura.

■ 4.3 Método

Técnica	O que responde	Detalhe
Correlação de Pearson (r)	Duas variáveis andam juntas em linha reta?	p-valor pela distribuição t com n − 2 graus de liberdade; intervalo de 95% pela transformação z de Fisher
Correlação de Spearman (ρ)	Andam juntas, mesmo que não em linha reta?	Pearson sobre os postos, com empates resolvidos pela média; confirma o Pearson quando há valores extremos
Regressão linear simples	Quanto o tempo muda a cada ponto do fator mais forte?	Mínimos quadrados; erro-padrão, teste t e intervalo de 95% para a inclinação; R^2
Regressão linear múltipla	O efeito continua quando os outros fatores são controlados?	Mínimos quadrados por eliminação de Gauss-Jordan; t, p e intervalo por coeficiente; beta padronizado; R^2 ajustado; teste F
Tendência semanal	O volume de denúncias está subindo?	Regressão do volume no tempo, com intervalo de predição de 95% para as próximas 4 semanas; se a inclinação não é significativa, a projeção usa a média
Dimensionamento	Quantas pessoas a demanda pede?	Demanda projetada dividida pela capacidade observada por pessoa

Uma regra de redação foi programada no servidor: uma relação só é chamada de relação quando p < 0,05. Abaixo disso, a tela diz "sem evidência de relação". O sistema nunca escreve "correlação zero" quando a verdade é "não deu para medir"; valores que não podem ser calculados chegam como nulos e aparecem como "indisponível".

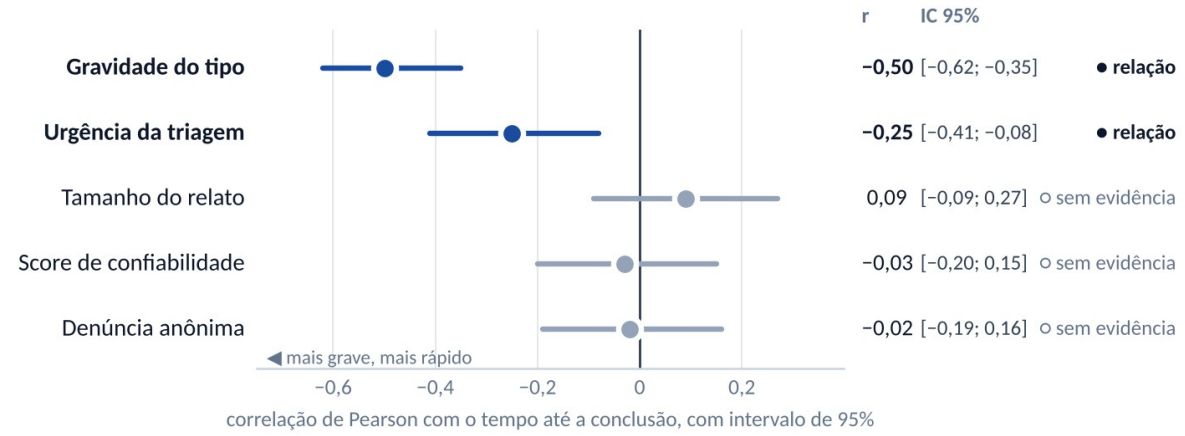
Validação do cálculo. As distribuições t e F foram implementadas a partir da beta incompleta regularizada, sem biblioteca externa, em `util/Inferencia.java`. Para provar que estão certas, o teste unitário usa o quarteto de Anscombe (1973), cujos resultados são conhecidos: r = 0,8164, p = 0,00217, reta y = 3,0001 + 0,5001x e F = 17,99. Os quatro conferem, e outros testes cobrem regressão múltipla exata, matriz singular, empates no Spearman e série constante.

■ 4.4 Resultados

▸ 4.4.1 O que se relaciona com o tempo de atendimento

▼ **Tabela 10.** Correlação de cada fator com o tempo até a conclusão (n = 123).

Fator	Pearson r	IC 95%	p-valor	Spearman ρ	Leitura
Gravidade do tipo	−0,50	[−0,62; −0,35]	< 0,001	−0,50	**Significativa**, moderada: quanto mais grave, mais rápido
Urgência da triagem	−0,25	[−0,41; −0,08]	0,005	−0,22	**Significativa**, fraca
Tamanho do relato	0,09	[−0,09; 0,27]	0,303	0,13	Sem evidência de relação
Score de confiabilidade	−0,03	[−0,20; 0,15]	0,764	−0,03	Sem evidência de relação
Denúncia anônima	−0,02	[−0,19; 0,16]	0,852	0,07	Sem evidência de relação



▲ **Figura 8.** *Correlação de cada fator com o tempo de atendimento. Só as linhas que não cruzam o zero indicam relação.*



▲ **Figura 9.** *Correlação com o tempo e matriz de Pearson, na aba Estatística do painel. Em cinza, as relações sem significância.*

A matriz mostra ainda que gravidade e urgência andam muito juntas (r = 0,73): as regras do VigIA partem do tipo para atribuir a urgência. Por isso, na regressão múltipla, as duas disputam o mesmo efeito.

▸ 4.4.2 Regressão simples: quanto a gravidade encurta o prazo

O fator mais forte é a gravidade, e a reta ajustada é:

$$\text{tempo (dias)} = 12{,}245 - 1{,}583 \times \text{gravidade}$$

Cada ponto de gravidade encurta o atendimento em 1,58 dia, com intervalo de 95% entre 1,08 e 2,08 dias. Sozinha, a gravidade explica 25% da variação do tempo ($R^2 = 0{,}25$). Os outros 75% vêm de fatores que o sistema ainda não registra.

▸ 4.4.3 Regressão múltipla: o efeito resiste ao controle?

▼ **Tabela 11.** Regressão do tempo em todos os fatores juntos. $R^2 = 0,30$; R^2 ajustado = 0,27; F = 9,99 (p < 0,001); erro-padrão residual de 3,12 dias.

Variável	Coeficiente (dias)	IC 95%	p-valor	β padronizado
Intercepto	12,170	[8,712; 15,629]	< 0,001	-
Gravidade do tipo	**−1,986**	[−2,726; −1,247]	**< 0,001**	**−0,62**
Urgência da triagem	0,427	[−0,434; 1,288]	0,328	0,12
Score de confiabilidade	−0,003	[−0,047; 0,042]	0,900	−0,01
Denúncia anônima	−0,904	[−2,628; 0,821]	0,302	−0,09
Tamanho do relato	0,010	[−0,001; 0,021]	0,066	0,20

Com os outros fatores no modelo, só a gravidade continua significativa, e com efeito maior (β padronizado de −0,62). A urgência perde a significância porque, como mostrou a matriz, carrega quase a mesma informação que a gravidade.

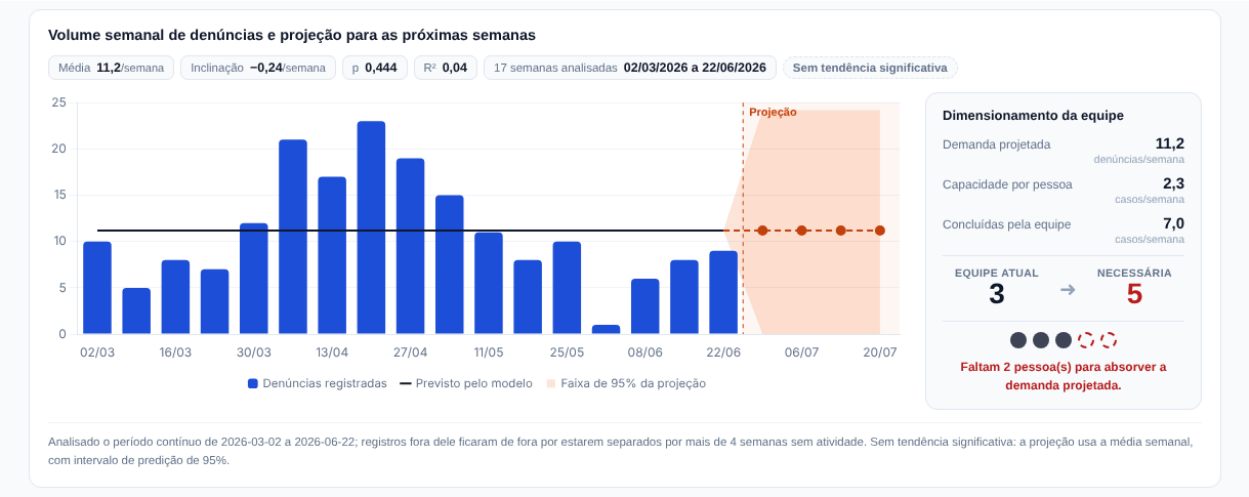


▲ **Figura 10.** *Regressão simples (dispersão com a reta) e regressão múltipla, como aparecem no painel.*

▸ 4.4.4 Demanda e equipe

O volume semanal foi analisado no período contínuo de 02/03 a 22/06/2026 (17 semanas). A inclinação é de −0,24 denúncia por semana, com p = 0,44: **não há tendência significativa**. Por isso a projeção das próximas 4 semanas usa a média, 11,2 denúncias por semana, com intervalo de predição de 95%. Projetar a reta de uma tendência que não existe é um erro comum que o servidor evita de propósito.

A equipe concluiu, em média, 7,0 casos por semana com 3 pessoas, ou 2,33 por pessoa. Para acompanhar 11,2 denúncias por semana, são necessárias **5 pessoas**. Hoje são 3.

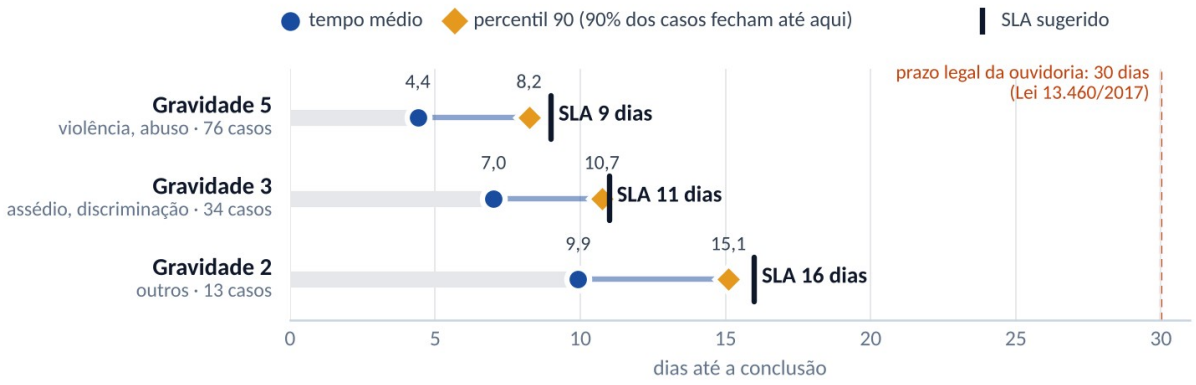


▲ **Figura 11.** *Volume semanal, projeção com faixa de 95% e dimensionamento da equipe.*

▸ 4.4.5 Prazo por gravidade, direto do banco

A consulta 3 do script `07_estatistica_fase7.sql` calcula, por gravidade, a média e o percentil 90 do tempo. O percentil 90 é o prazo que 90% dos casos cumprem, e por isso serve de base para um prazo de atendimento (SLA) honesto.

Gravidade	Casos	Tempo médio (dias)	Percentil 90 (dias)	SLA sugerido
5 (violência, abuso)	76	4,40	8,23	9 dias
3 (assédio, discriminação)	34	7,01	10,74	11 dias
2 (outros)	13	9,92	15,10	16 dias



▲ **Figura 12.** *Prazo por gravidade: o SLA sugerido fica logo acima do percentil 90, bem antes dos 30 dias da ouvidoria.*

■ 4.5 Do número à decisão

O servidor não devolve só números: devolve conclusões com nível (achado, alerta, ponto positivo, contexto), a evidência estatística e a decisão que ela sustenta. A tabela resume as cinco conclusões, e a figura seguinte mostra como aparecem no painel.

Achado	Evidência	Decisão sugerida
A gravidade é o que mais explica o prazo	r = −0,50; p < 0,001; IC da inclinação de −2,08 a −1,08 dias	Manter a fila por gravidade e criar SLA por tipo (9, 11 e 16 dias)
O score de confiabilidade não prevê nada	r = −0,03; p = 0,76	Trocar o score por um de risco, baseado no Formulário Nacional de Avaliação de Risco (FONAR, Lei 14.149/2021), com confirmação humana registrada na auditoria
Denunciar anonimamente não atrasa o caso	r = −0,02; p = 0,85	Manter o anonimato como padrão e divulgá-lo como garantia do canal
O modelo explica só 27% da variação	R^2 ajustado = 0,27	Registrar o órgão de destino e a carga da equipe em cada caso, para a próxima versão explicar mais
A demanda pede 5 pessoas; há 3	Média de 11,2/semana; 2,33 casos por pessoa	Reforçar a equipe ou redistribuir casos antes que a fila cresça



▲ **Figura 13.** *Cartões de conclusão no painel: cada um traz a evidência e a decisão sugerida.*

> ★ **O ACHADO MAIS IMPORTANTE PARA O PRODUTO**
>
> O score de confiabilidade existe desde a Fase 2 e aparece em todas as telas, mas mede se o formulário foi bem preenchido, e não o risco da vítima. A estatística mostrou que ele **não se relaciona com nada que importa no atendimento**.

■ 4.6 Onde está no sistema

Peça	Arquivo
Cálculo (correlação, regressão, distribuições t e F)	`backend/.../util/Inferencia.java`
Análise, tendência, dimensionamento e conclusões	`backend/.../service/AnalisePreditivaService.java`
Endpoint (ADMIN, GESTOR, ATENDENTE e AUDITOR)	`GET /api/stats/regressao`
Tela	Aba Estatística, seção "Correlação e regressão", em `js/estatistica.js` e `css/regressao.css`
Consultas equivalentes no Oracle	`database/07_estatistica_fase7.sql`: `CORR`, `CORR_S`, `REGR_SLOPE`, `REGR_INTERCEPT`, `REGR_R2`, `PERCENTILE_CONT`
Resultados conferidos	`database/07_resultados_fase7.txt`: os números do banco batem com os da API
Testes	`InferenciaTest` (7) e `AnalisePreditivaServiceTest` (5)

```
-- 07_estatistica_fase7.sql, consulta 2: a regressão inteira sai do banco
SELECT ROUND(REGR_INTERCEPT(tempo_dias, gravidade), 3) AS intercepto_a,
       ROUND(REGR_SLOPE(tempo_dias, gravidade), 3)     AS inclinacao_b,
       ROUND(REGR_R2(tempo_dias, gravidade), 3)        AS r2,
       REGR_COUNT(tempo_dias, gravidade)               AS n
  FROM VW_ANALISE_TEMPO;
-- resultado: 12,245 | -1,583 | 0,247 | 123  (igual ao da API)
```

■ 4.7 Cuidados na leitura

- **Correlação não é causalidade.** Casos graves fecham mais rápido porque a fila os prioriza, e não porque a gravidade, por si, acelere algo. É justamente isso que a análise confirma: a priorização está funcionando.
- **Dados sintéticos.** Com dados reais, os números mudam; o método e a tela continuam valendo.
- **Multicolinearidade.** Gravidade e urgência carregam a mesma informação. O beta padronizado da regressão múltipla ajuda a comparar os efeitos, mas a urgência não deve ser lida como "inútil": ela só não acrescenta nada além do tipo.
- **Pressupostos.** A regressão linear supõe resíduos aproximadamente normais e de variância constante. Com tempos sempre positivos e assimetria à direita, um modelo de sobrevivência (Cox) seria o passo seguinte, e permitiria usar também os casos ainda abertos.

■ 4.8 Síntese da Parte 3

Pedido	Onde foi atendido
✔️ Correlação	4.4.1: Pearson e Spearman com p-valor e intervalo de confiança; matriz entre fatores
✔️ Regressão	4.4.2 a 4.4.4: simples, múltipla e tendência com projeção
✔️ Apoiar decisão	4.5: cinco conclusões, cada uma com a evidência e a decisão sugerida, também no painel
✔️ Relatório	Aba Estatística do painel e consultas no banco, com resultados conferidos

⬢ PARTE 4: Blockchain e Smart Contract

✎ **O QUE A ATIVIDADE PEDE**

Aplicar blockchain ao projeto.

Construir um protótipo de smart contract em Solidity.

■ 5.1 O problema que a blockchain resolve aqui

Blockchain não entrou no Protege+ por ser tendência. Ela entrou porque o exercício B da Parte 1 deixou um problema sem solução dentro do servidor.

A trilha de auditoria é encadeada por SHA-256: cada registro carrega o hash do anterior. Isso detecta qualquer alteração ou remoção **no meio** da trilha. Mas, quando um backup antigo é restaurado, a trilha é **cortada no fim**, e o que sobra continua coerente consigo mesmo. No exercício, 18 registros sumiram e a verificação interna respondeu "cadeia intacta".

O mesmo vale para um administrador mal-intencionado que apague os últimos registros, ou para um ransomware que force a volta a uma cópia velha. É preciso uma testemunha fora do servidor, que ninguém do órgão consiga reescrever, e que qualquer fiscal consiga consultar.

★ **A IDEIA EM UMA FRASE**

Quem guarda a cadeia não pode ser quem atesta que ela está completa. A blockchain é a testemunha que fica fora do servidor.

A mesma testemunha resolve um segundo problema: provar, diante de um juiz, que a foto de uma lesão é exatamente a que foi entregue, e desde quando.

■ 5.2 Por que blockchain, e não outra coisa

▼ **Tabela 12.** Alternativas avaliadas para guardar a prova fora do servidor.

Alternativa	Vantagem	Por que não basta sozinha
Servidor de log central do próprio órgão	Simples, já existe em muitos órgãos	O mesmo órgão administra os dois lados: não resolve a fraude interna
Armazenamento imutável (WORM, object lock)	Impede apagar por um prazo	Quem prova ao fiscal externo que a política estava ligada? A confiança volta para o administrador
Publicar o hash no Diário Oficial	Público e com valor legal	Diário, manual e lento; não dá para ancorar de hora em hora
Carimbo de tempo ICP-Brasil (RFC 3161)	Valor legal forte para um documento	Cada carimbo é um recibo guardado pelo próprio órgão; omitir o último recibo esconde o corte
Blockchain com contrato inteligente	Registro público e ordenado, que só cresce; a regra "nunca voltar atrás" é imposta pelo código; qualquer participante confere	Exige rede e cuidado com dado pessoal (5.6)

A diferença decisiva é a **lista ordenada e pública**. Com recibos de carimbo de tempo, o fiscal só vê os recibos que o órgão mostra. Na blockchain, ele vê todos os checkpoints, e o contrato recusa qualquer ancoragem que volte no

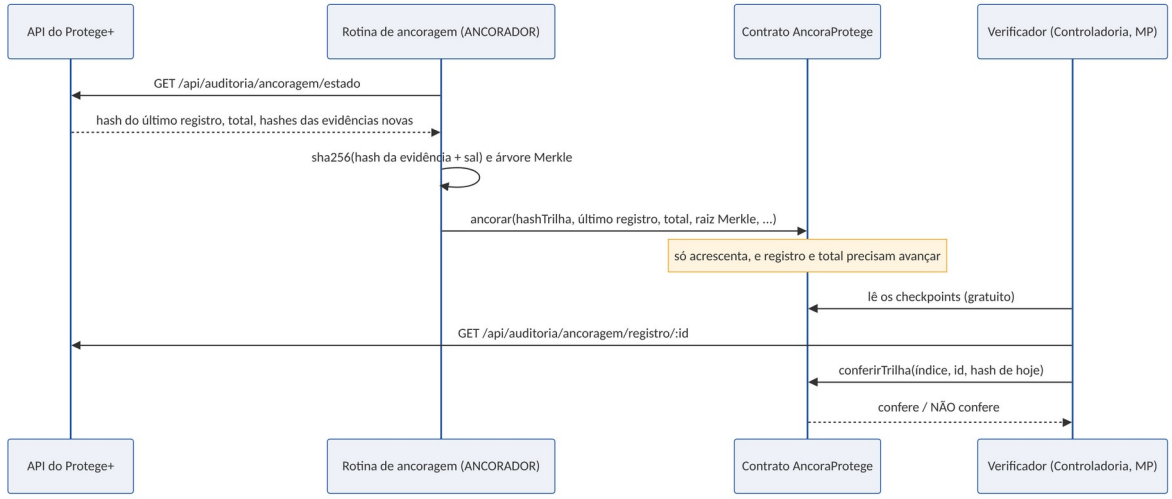
tempo. O carimbo ICP-Brasil continua útil, e complementar: para levar uma evidência a juízo, os dois juntos são mais fortes do que cada um.

■ 5.3 Que blockchain

Rede	Custo por ancoragem	A favor	Contra
Rede Blockchain Brasil (RBB), criada em 2022 por BNDES e TCU, baseada em Hyperledger Besu	Sem taxa de mercado	Feita para o setor público; os fiscais (como o TCU) já participam; compatível com Solidity sem mudar uma linha	Depende da adesão do órgão
Rede pública de segunda camada (ex.: Polygon)	Centavos	Qualquer pessoa confere, sem pedir acesso	Custo em criptomoeda, que oscila
Ethereum principal	Alguns reais ou mais, conforme a rede	Máxima independência	Caro para ancorar de hora em hora

A recomendação é a RBB. O contrato foi compilado para a versão `paris` da máquina virtual Ethereum justamente para rodar em redes Besu mais conservadoras. Para a demonstração, usa-se a rede local do Hardhat, que se comporta como a Ethereum.

■ 5.4 O contrato AncoraProtege



▲ **Figura 14.** *Fluxo de ancoragem e de verificação. O servidor não fala com a blockchain: quem ancora e quem confere são rotinas externas.*

▸ 5.4.1 O que vai para a blockchain

Cada **checkpoint** guarda: o hash do último registro da trilha (que, pelo encadeamento, resume todos os anteriores), o número desse registro, o total de registros, a raiz Merkle de um lote de evidências, a data do bloco e a conta que ancorou. Ancorar uma vez por hora significa que um corte na trilha nunca pode esconder mais do que uma hora de registros.

Evidências. O hash puro de um arquivo **nunca** vai para a cadeia. Se fosse, quem tem uma cópia da foto (o próprio agressor, que muitas vezes tirou a foto ou tem o celular da vítima) poderia calcular o hash e descobrir que ela foi entregue como prova. Cada evidência entra como um compromisso: `sha256(hash do arquivo + sal)`, com

um sal aleatório de 32 bytes que fica só com o órgão. Os compromissos do lote viram uma árvore Merkle, e só a raiz vai para a cadeia. Ancorar 100 evidências custa o mesmo que ancorar uma.



▲ **Figura 15.** *Como uma foto vira prova sem se expor: só a raiz da árvore vai para a blockchain, e o sal nunca sai do órgão.*

▸ 5.4.2 Papéis

Papel	Quem	Pode	Não pode
Gestor	O órgão	Autorizar e revogar ancoradores; contestar uma ancoragem	Ancorar
Ancorador	A rotina automática de ancoragem	Acrescentar checkpoints	Alterar ou apagar qualquer coisa; contestar
Qualquer pessoa	Fiscal, Ministério Público, juiz	Ler e conferir, sem custo	Escrever

A separação de funções da Parte 1 se repete aqui. Se a chave do servidor vazar, o gestor revoga o ancorador na hora. E se o invasor tiver usado a chave para ancorar algo falso, o gestor marca aquela ancoragem como **contestada**: ela continua visível para sempre (a fraude também é registro), mas deixa de provar qualquer coisa e deixa de servir de referência para as próximas.

▸ 5.4.3 Funções

Função	Quem chama	O que faz
`ancorar(...)`	Ancorador	Grava um checkpoint. Recusa se o registro, o total ou a última evidência voltarem em relação ao último checkpoint válido
`contestar(indice, motivo)`	Gestor	Marca um checkpoint como contestado, sem apagá-lo, e recalcula qual é o último válido
`conferirTrilha(indice, registro, hash)`	Qualquer um (leitura)	Diz se o registro de hoje tem o mesmo hash que foi ancorado
`provarEvidencia(indice, compromisso, prova)`	Qualquer um (leitura)	Confere a prova Merkle de uma evidência
`checkpoint(i)`, `ultimoValido()`, `totalCheckpoints()`	Qualquer um (leitura)	Consulta

▸ 5.4.4 O coração do contrato

O trecho abaixo é a regra que resolve o exercício B: um servidor restaurado de um backup antigo não consegue "reancorar" o passado.

```
function ancorar(bytes32 hashTrilha, uint64 ultimoRegistro, uint64 totalRegistros,
                 bytes32 raizEvidencias, uint64 ultimaEvidencia, uint32 quantidadeEvidencias)
    external onlyRole(ANCORADOR) returns (uint256 indice)
{
    if (hashTrilha == bytes32(0)) revert HashVazio();
    if (ultimoRegistro == 0 || totalRegistros == 0 || totalRegistros > ultimoRegistro)
        revert DadosIncoerentes();
    if ((quantidadeEvidencias == 0) != (raizEvidencias == bytes32(0)))
        revert LoteIncoerente();

    if (_ultimoValido != 0) {
        Checkpoint storage ant = _checkpoints[_ultimoValido - 1];
        if (ultimoRegistro <= ant.ultimoRegistro)
            revert RegistroNaoAvancou(ant.ultimoRegistro, ultimoRegistro);
        if (totalRegistros <= ant.totalRegistros)
            revert TotalNaoAvancou(ant.totalRegistros, totalRegistros);
        // ... a ultima evidencia tambem nunca volta
    }
    indice = _checkpoints.length;
    _checkpoints.push(Checkpoint({ hashTrilha: hashTrilha, /* ... */
        ancoradoEm: uint40(block.timestamp), ancorador: msg.sender }));
    _ultimoValido = indice + 1;
    emit CheckpointAncorado(indice, ultimoRegistro, totalRegistros, hashTrilha, ...);
}
```

O contrato usa duas bibliotecas auditadas do OpenZeppelin, `AccessControl` para os papéis e `MerkleProof` para as provas, em vez de reimplementar criptografia. Os erros são personalizados (`RegistroNaoAvancou`, `JaContestado`...), mais baratos que mensagens em texto e mais fáceis de testar. O contrato não tem função `payable`: ele recusa qualquer envio de dinheiro.

■ 5.5 Testes do contrato

São 21 testes automatizados (Hardhat, Mocha e Chai), com cobertura de 100% das linhas e 90,9% dos desvios do contrato.

Grupo	O que comprova
Implantação e papéis (5)	Gestor não ancora; conta estranha não ancora nem contesta; ancorador revogado para na hora; gestor nulo é recusado; o contrato não aceita dinheiro
Ancoragem da trilha (5)	Checkpoint gravado com data do bloco e evento; dados incoerentes recusados; **servidor restaurado não reancora o passado**; a história inteira continua consultável
Evidências (6)	Prova Merkle de cada foto do lote; **um pixel alterado muda o hash e a prova falha**; prova de um lote não vale em outro; lote não volta; **o hash puro da foto não aparece em lugar nenhum da cadeia**
Contestação (4)	Ancoragem forjada com número gigante deixa de travar as próximas depois de contestada; contestar duas vezes é recusado; contestações em sequência voltam ao último válido
Custo (1)	Ancorar a trilha com 100 evidências custa menos de 200 mil de gas; a prova de uma evidência tem no máximo 7 hashes

O teste de privacidade merece destaque: ele monta um lote de cinco fotos, ancora, e procura o hash de cada arquivo em todos os dados públicos da transação e dos eventos. Não encontra. Encontra só a raiz do lote.

■ 5.6 Blockchain e LGPD

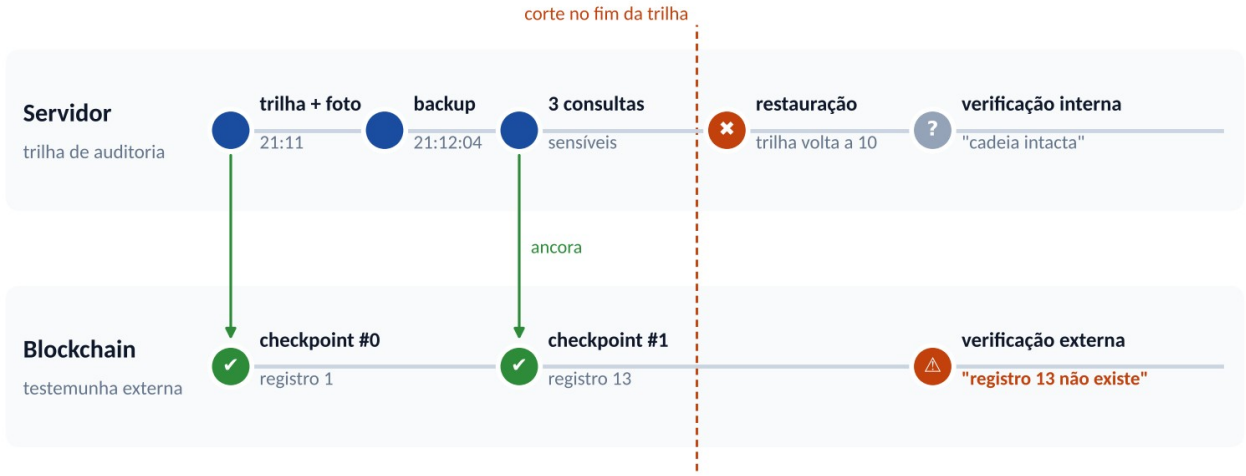
Um registro em blockchain não se apaga, e a LGPD garante a eliminação de dados pessoais quando a finalidade acaba (art. 16 e art. 18, VI). A conciliação está no desenho, e não numa exceção.

- **Nada pessoal vai para a cadeia.** Só hashes, contadores e datas. Nenhum nome, protocolo, relato, endereço, e-mail ou IP.
- **Evidências entram com sal.** Sem o sal, o compromisso não revela nada nem para quem tem o arquivo.
- **Eliminar é apagar o sal.** Quando um caso é excluído, apagar o sal das evidências dele torna o compromisso na cadeia definitivamente impossível de ligar a qualquer arquivo. A técnica é conhecida como apagamento criptográfico.
- **A trilha ancorada já é anonimizada.** O hash resume registros que, por decisão da Fase 5, nunca copiam o relato.

■ 5.7 Demonstração e exercício C

A demonstração completa rodou contra a API real e uma rede local, e está gravada em `blockchain/resultados/demonstracao-20261027.txt`. O destaque é o **exercício C**, que repete o exercício B da Parte 1, agora com a testemunha externa.

1. Uma denúncia anônima recebe uma foto. O contrato é implantado, a trilha e a foto são ancoradas.
2. A foto baixada do sistema é provada contra a cadeia: *é a evidência #1, sem nenhuma alteração, ancorada em 27/10/2026, 21:11*. A mesma foto com um pixel trocado é recusada.
3. O ADMIN gera um backup. Depois dele, uma atendente abre três casos: três consultas sensíveis entram na trilha. A rotina ancora de novo (registro 13).
4. O banco é perdido e o backup é restaurado, com o hash conferido contra o manifesto.
5. A verificação interna diz que está tudo certo. A blockchain diz que não.



A cadeia interna não percebe o corte no fim. A blockchain percebe, porque guardou o registro 13.

▲ **Figura 16.** *O exercício C em duas faixas: o corte no fim da trilha passa pela verificação interna, mas não pela blockchain.*

```
blockchain/resultados/demonstracao-20261027.txt (trecho)
[21:12:34] 5. API de volta. A verificacao INTERNA da cadeia nao ve problema nenhum:
   trilha: 8 registros, integra = True | Cadeia intacta: todos os elos conferem
[21:12:35] 6. A rotina de ancoragem tenta rodar de novo:
  [ALERTA] O checkpoint #1 (registro 13, 27/10/2026, 21:12:05) nao confere mais.
           A trilha foi cortada ou alterada desde a ultima ancoragem. Nada foi ancorado.
[21:12:36] 7. Verificacao independente contra a blockchain:
  [OK]     #0  27/10/2026, 21:11:18  registro 1 presente, hash confere
  [ALERTA] #1  27/10/2026, 21:12:05  o registro 13 NAO EXISTE MAIS na trilha
               Ancorado com 13 registros; hoje a trilha tem 10.
  [ALERTA] 1 checkpoint(s) nao conferem. A trilha atual NAO e a mesma que foi ancorada.
```

As três consultas sensíveis da atendente sumiram do servidor sem deixar marca, mas não da história: a blockchain prova que elas existiram, quando, e que a trilha de hoje não é a mesma. A rotina de ancoragem também se recusa a carimbar a trilha cortada, porque ancorar por cima dela seria dar o selo de autenticidade justamente à versão adulterada.

■ 5.8 Custos medidos

Operação	Gas	Observação
Implantar o contrato	1.112.475	Uma vez
Ancorar só a trilha	110.891	A cada hora
Ancorar a trilha e um lote de evidências	cerca de 162.500	O mesmo com 2 ou com 100 evidências
Conferir trilha ou provar evidência	0	Leitura, sem transação

Na RBB, esse gas não tem preço de mercado. Numa rede pública, 24 ancoragens por dia ficam na casa dos centavos por dia numa rede de segunda camada.

■ 5.9 Limites declarados

⚠ **O conteúdo perdido não volta pela blockchain.** Ela prova que os registros existiam e qual era o hash; o texto só volta de um backup mais recente ou da réplica (Parte 2).

⚠️ **O verificador pergunta à API qual é o hash de hoje.** Um servidor totalmente comprometido poderia mentir nessa resposta. A defesa completa é o auditor receber uma cópia da trilha e recalcular a cadeia por conta própria, comparando com a blockchain.

⚠️ **Os sais ficam numa pasta da rotina de ancoragem**, fora do Git. Sem eles não há prova de evidência: a pasta precisa entrar na política de backup.

⚠️ **A demonstração usa uma rede local.** A adesão à RBB e a operação de um nó são passos institucionais, não de código.

■ 5.10 Síntese da Parte 4

Pedido	Onde foi atendido
✔️ Aplicar blockchain ao projeto	5.1 a 5.3: o problema real (trilha cortada), quatro alternativas comparadas, escolha da rede
✔️ Protótipo de smart contract em Solidity	5.4: `blockchain/contracts/AncoraProtege.sol`, com papéis, contestação e prova Merkle
✔️ Comprovação	5.5 (21 testes) e 5.7 (exercício C contra a API real)
✔️ Privacidade	5.6: nada pessoal na cadeia, compromissos com sal, apagamento criptográfico

★ 6. Conclusão

■ 6.1 O que foi entregue

A pergunta desta entrega era se o Protege+ merece a confiança de quem denuncia e de quem fiscaliza. No começo da fase, a resposta honesta era "ainda não": a senha de administrador estava no JavaScript público e o segredo dos tokens estava no repositório. No fim, cada defesa foi atacada e o resultado está gravado.

Parte	Entrega	Comprovação
1. Segurança	26 falhas corrigidas; políticas de autenticação, criptografia, backup e acesso escritas a partir do código; backup verificado por hash	11 ataques bloqueados; adulteração detectada; banco restaurado em 10 s
2. Redes	Topologia por zonas; borda nginx com HTTPS, TLS mútuo para aparelhos, limites e página de queda	39 verificações; painel inteiro por HTTPS; 5 defeitos achados pelo teste
3. Estatística	Correlação, regressão simples e múltipla, tendência e dimensionamento, com conclusões no painel	Cálculo conferido contra Anscombe; SQL igual à API
4. Blockchain	Contrato AncoraProtege, rotinas de ancoragem, verificação e prova de evidência	21 testes; exercício C detectou o corte da trilha

■ 6.2 O que não foi feito

- ⚠ Segundo fator de autenticação e login gov.br para servidores.
- ⚠ Cifra de verdade no armazenamento do navegador (hoje é ofuscação, declarada na seção 2.4.2).
- ⚠ Montagem física da topologia de produção: zonas de rede, VPN, réplica do banco e mitigação de DDoS.
- ⚠ Adesão à Rede Blockchain Brasil e operação da ancoragem horária.
- ⚠ Avaliação com usuárias reais e com profissionais da rede de proteção.

■ 6.3 Próximos passos, por prioridade

#	Passo	Por quê
1	Trocar o score de confiabilidade por um score de risco baseado no FONAR, com confirmação humana	A Parte 3 mostrou que o score atual não prevê nada
2	Login gov.br com segundo fator para servidores	Senha sozinha é o vetor de ataque mais comum contra órgãos públicos
3	Bibliotecas do front-end servidas pelo próprio servidor ou com verificação de integridade (SRI)	Uma CDN comprometida hoje rodaria código no painel
4	Web Crypto (AES-GCM) no armazenamento do navegador e reenvio automático de denúncias feitas sem rede	Fecha as duas lacunas declaradas do modo offline
5	Ancoragem horária na RBB e o auditor recalculando a trilha por conta própria	Fecha o limite declarado em 5.9
6	Modelo de sobrevivência (Cox) para o tempo de atendimento, usando também os casos abertos	A regressão linear ignora quem ainda espera
7	Avaliação com usuárias e profissionais da rede	Nenhuma das seis fases ouviu quem vai usar o canal

■ 6.4 Observação de método

> ★ **PRIMEIRO O ATAQUE, DEPOIS O TEXTO**
>
> O padrão que atravessou as fases anteriores era o documento prometer mais do que o código entregava. Esta fase inverteu a ordem.

O efeito mais visível não foi a lista de defesas, e sim a lista de defeitos que só apareceram quando alguém tentou quebrar o sistema. É esse hábito, mais do que qualquer tecnologia, que pode ser levado para um órgão público de verdade.

☰ Referências

▸ Legislação

- BRASIL. Lei nº 11.340, de 7 de agosto de 2006 (Lei Maria da Penha).
- BRASIL. Lei nº 13.709, de 14 de agosto de 2018 (Lei Geral de Proteção de Dados Pessoais).
- BRASIL. Lei nº 14.149, de 5 de maio de 2021 (Formulário Nacional de Avaliação de Risco).

▸ Segurança e redes

- NIST. The NIST Cybersecurity Framework (CSF) 2.0. NIST CSWP 29, 2024.
- NIST. SP 800-207: Zero Trust Architecture, 2020.
- NIST. SP 800-52 Rev. 2: Guidelines for the Selection, Configuration, and Use of TLS Implementations, 2019.
- IETF. RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3, 2018.
- IETF. RFC 6797: HTTP Strict Transport Security (HSTS), 2012.
- IETF. RFC 3161: Time-Stamp Protocol (TSP), 2001.
- OWASP. OWASP Top 10: 2021 e Application Security Verification Standard (ASVS) 4.0.3.
- ISO/IEC 27001:2022. Information security management systems.
- GOV.UK Design System. Exit this page (padrão de saída rápida).

▸ Estatística

- ANSCOMBE, F. J. Graphs in Statistical Analysis. The American Statistician, v. 27, n. 1, p. 17-21, 1973.
- FISHER, R. A. On the probable error of a coefficient of correlation deduced from a small sample. Metron, v. 1, p. 3-32, 1921.
- MONTGOMERY, D. C.; PECK, E. A.; VINING, G. G. Introduction to Linear Regression Analysis. 6. ed. Wiley, 2021.
- PRESS, W. H. et al. Numerical Recipes: The Art of Scientific Computing. 3. ed. Cambridge, 2007 (beta incompleta e função gama).

▸ Blockchain

- NAKAMOTO, S. Bitcoin: A Peer-to-Peer Electronic Cash System, 2008.
- MERKLE, R. C. A Digital Signature Based on a Conventional Encryption Function. CRYPTO 1987.
- WOOD, G. Ethereum: A Secure Decentralised Generalised Transaction Ledger (Yellow Paper).
- OPENZEPPELIN. Contracts 5.x: AccessControl e MerkleProof. Documentação oficial.
- Valora Analitik: Brasil lanza red blockchain para monitorear gasto público, 31/05/2022 (lançamento da RBB por BNDES e TCU, em Hyperledger Besu).

▸ Contexto

- FÓRUM BRASILEIRO DE SEGURANÇA PÚBLICA. Anuário Brasileiro de Segurança Pública 2026.
- MINISTÉRIO DAS MULHERES. Balanço do Ligue 180, 2025.

✎ Anexo A. Rotas novas da API

Método e rota	Perfis	Finalidade
`POST /api/auth/demo/perfil`	ADMIN, só em demonstração	Trocar de perfil na demonstração sem senha no cliente (auditado)
`GET /api/backup`	ADMIN, AUDITOR	Listar backups com o hash registrado e a situação (íntegro ou alterado)
`POST /api/backup`	ADMIN	Gerar backup manual (intervalo mínimo de 10 min)
`GET /api/stats/regressao`	Todos os perfis	Correlação, regressão, tendência, equipe e conclusões
`GET /api/auditoria/ancoragem/estado`	ADMIN, AUDITOR	Hash do último registro, total e hashes das evidências novas (auditado)
`GET /api/auditoria/ancoragem/registro/{id}`	ADMIN, AUDITOR	Hash atual de um registro já ancorado

✎ Anexo B. Como reproduzir cada resultado

Todos os comandos partem da pasta do repositório. O back-end roda em Windows com `build.cmd`; os roteiros de rede rodam em Linux ou WSL.

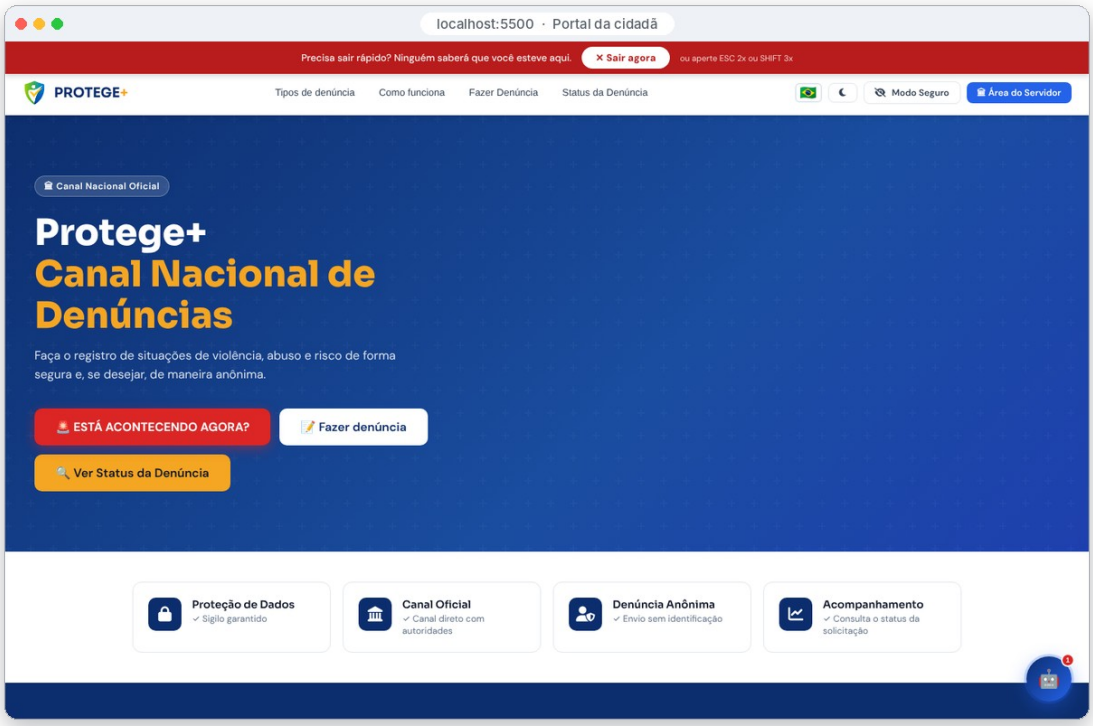
Resultado	Comando
Testes Java (217)	`cd backend` e `.\build.cmd test`
Subir a API	`cd backend` e `.\build.cmd run`
Simulação de ataques (Parte 1)	`py seguranca\simular_ataques.py`
Exercícios A e B (Parte 1)	`bash ../seguranca/exercicios_insider_e_recuperacao.sh target/protege-backend-1.0.0.jar` (Git Bash, na pasta backend, com a API parada)
Teste da borda de rede (Parte 2)	`bash infra/testar-rede.sh` (API com `PROTEGE_PROXY_CONFIAVEL=true` e `CORS_ORIGENS=https://localhost:8443`)
Consultas estatísticas (Parte 3)	`database/07_estatistica_fase7.sql` no Oracle; resultados em `07_resultados_fase7.txt`
Testes do contrato (Parte 4)	`cd blockchain`, `npm install` e `npm test`
Ancoragem e verificação (Parte 4)	`npm run no` em um terminal; `npm run implantar`, `npm run ancorar` e `npm run verificar` em outro
Tudo de uma vez, numa máquina limpa	Automático a cada push no GitHub Actions (`.github/workflows/provas.yml`): testes Java, testes do contrato, 11 ataques e 39 verificações de rede

✎ Anexo C. Arquivos novos desta entrega

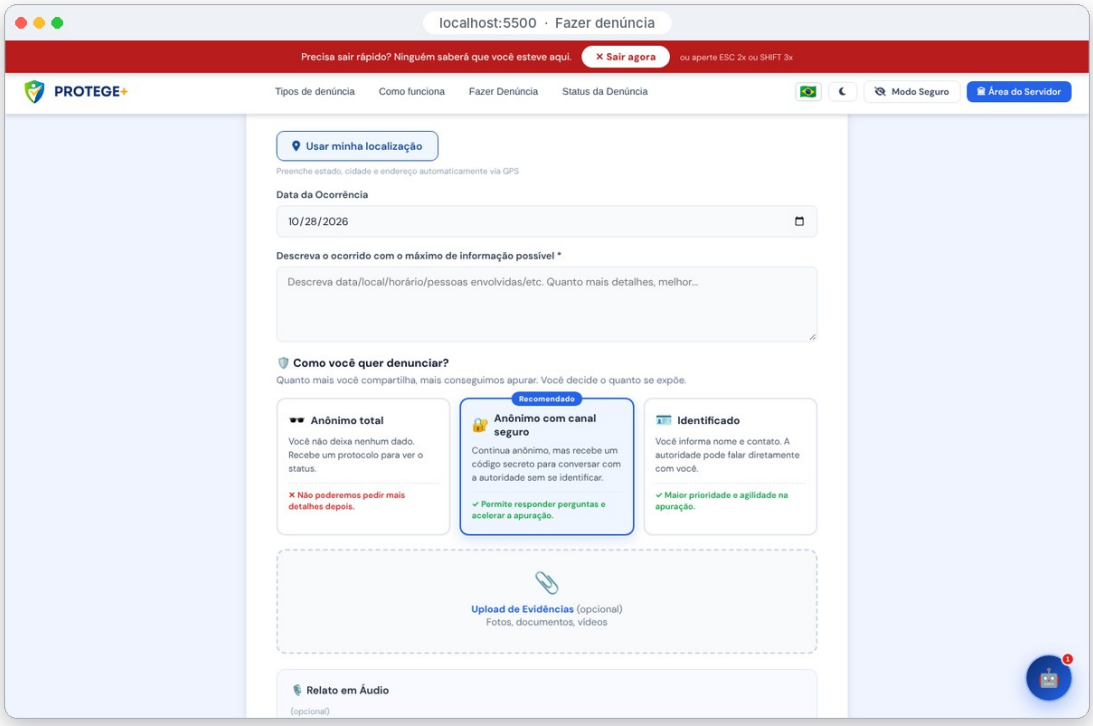
Pasta ou arquivo	Conteúdo
`backend/.../config/GuardiaoDeSegredos.java`	Segredos fortes ou sorteados; modo produção
`backend/.../security/` (filtros novos)	Limite de corpo, chave de dispositivo, limite por origem revisto
`backend/.../service/BackupService.java` e `FluxoStatus.java`	Backup verificado; máquina de estados do caso
`backend/.../util/Inferencia.java` e `AnalisePreditivaService.java`	Estatística inferencial (Parte 3)
`backend/.../controller/AncoragemController.java`	Dados para a ancoragem (Parte 4)
`backend/restaurar-backup.ps1` e `.cmd`	Restauração com conferência de hash
`database/06_ddl_fase7.sql`, `07_estatistica_fase7.sql`, `07_resultados_fase7.txt`	Restrições novas e consultas estatísticas
`js/storage-guard.js` (funções de escape)	Proteção contra XSS em todo o front-end
`css/regressao.css`	Tela de correlação e regressão
`seguranca/`	Simulação de ataques, exercícios e resultados (Parte 1)
`infra/`	Borda nginx, autoridade certificadora, teste de rede e resultados (Parte 2)
`blockchain/`	Contrato, testes, rotinas e resultados (Parte 4)
`.github/workflows/provas.yml`	Testes e provas automáticas a cada push (GitHub Actions)
`docs/img/telas/`	Telas do sistema rodando com a API (Anexo D)

✎ Anexo D. Telas do sistema

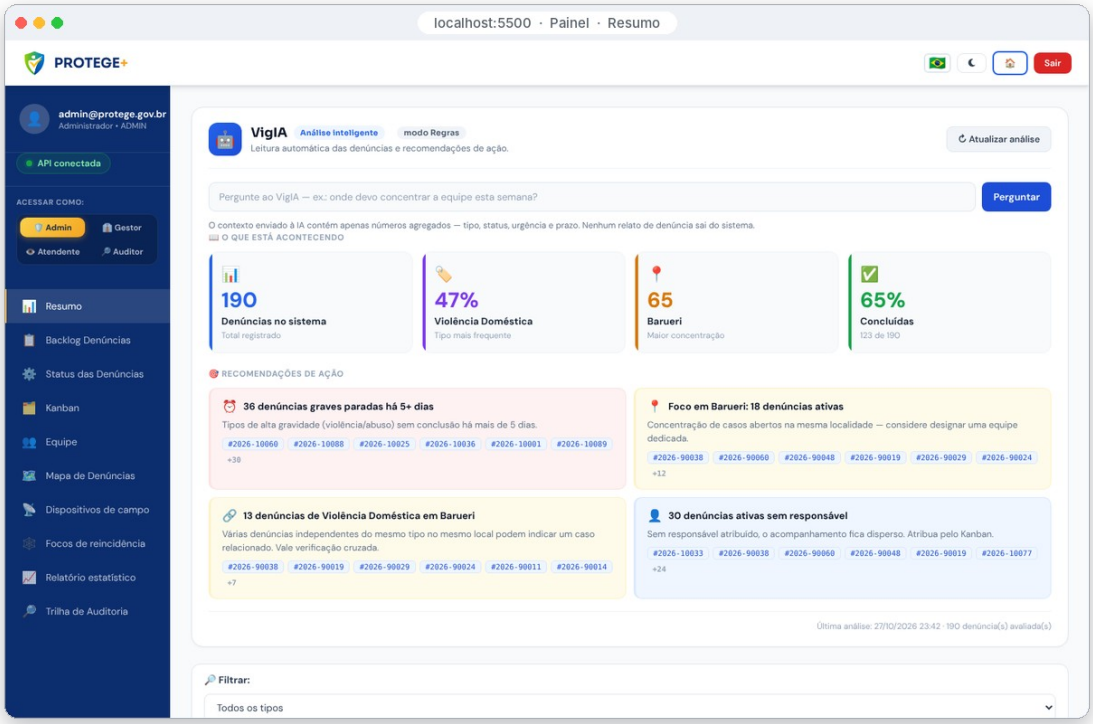
Telas capturadas com o sistema completo no ar (API Java e site), com o perfil ADMIN, sobre a base de demonstração.



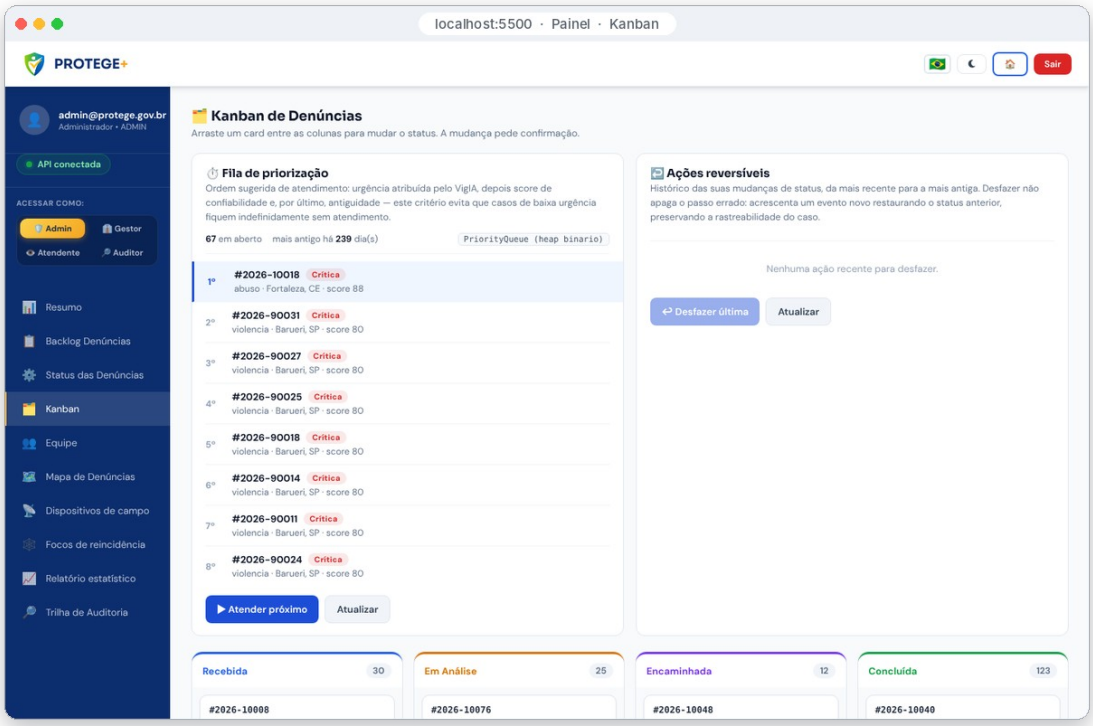
▲ **Figura 17.** *Portal da cidadã: denúncia sem login, com o botão de saída rápida sempre no topo.*



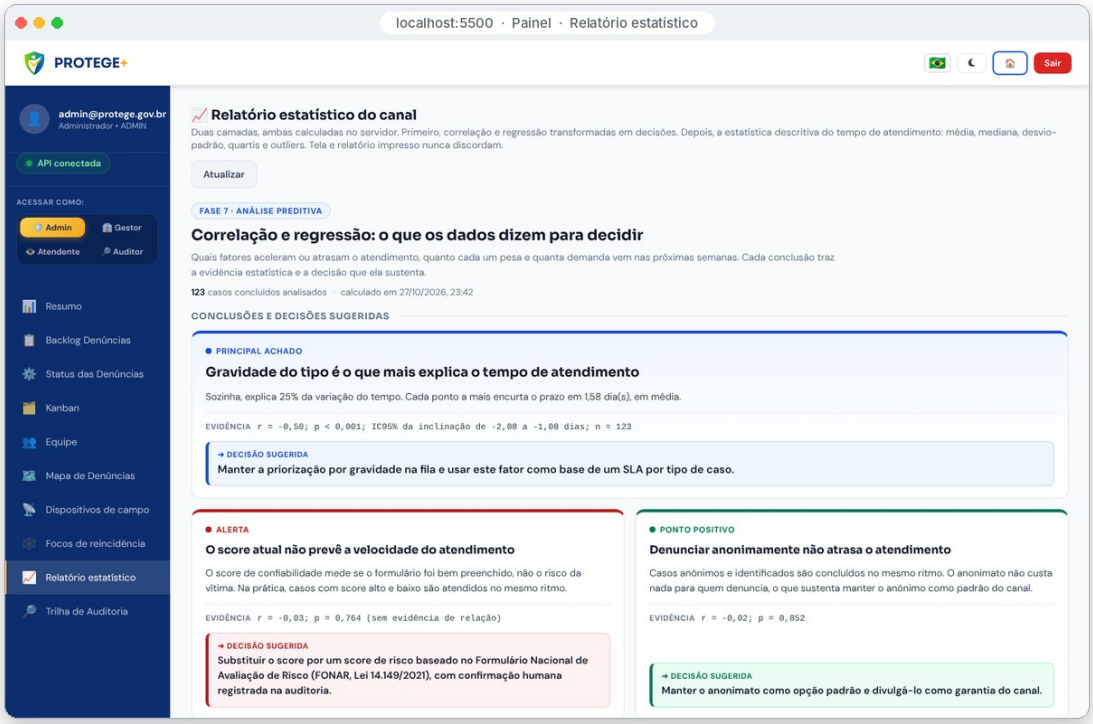
▲ **Figura 18.** *Formulário de denúncia: os três níveis de anonimato, com o que se ganha e o que se perde em cada um.*



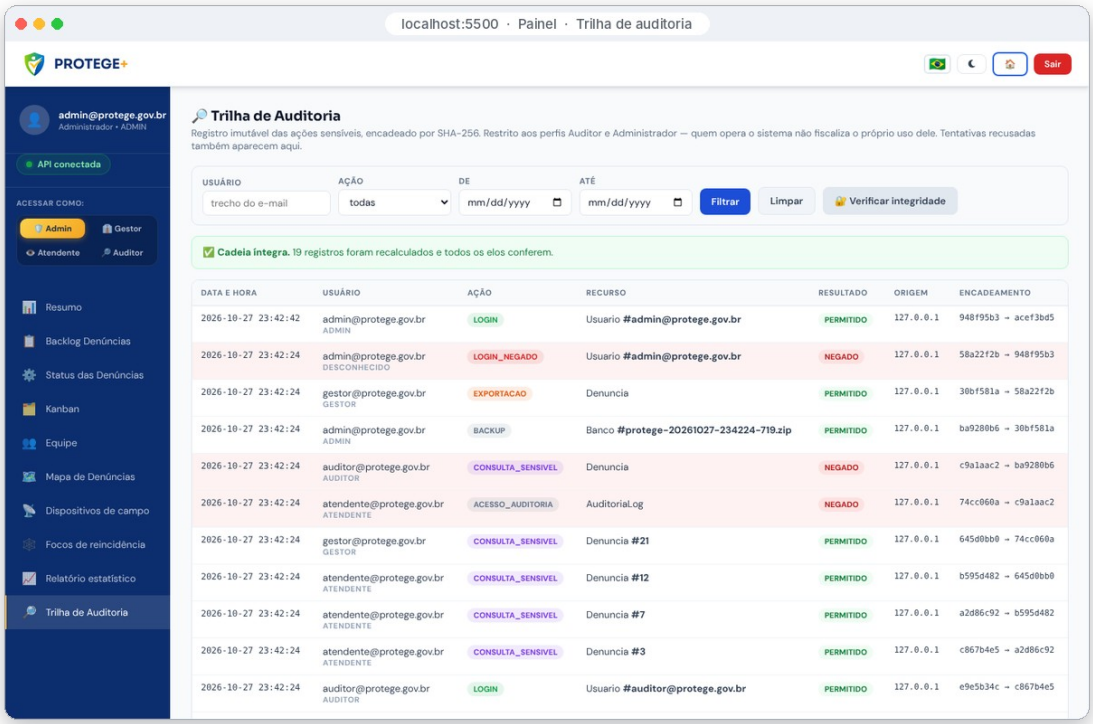
▲ **Figura 19.** *Painel do órgão: indicadores e recomendações do VigIA, sem enviar relato à IA.*



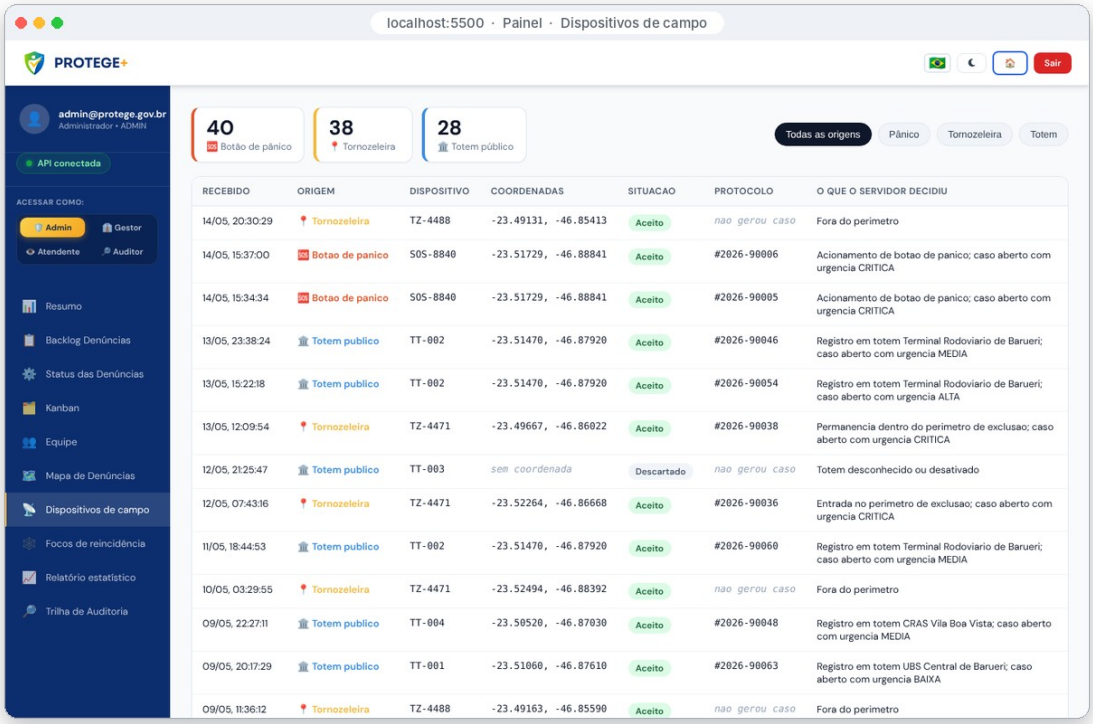
▲ **Figura 20.** *Fila de priorização e Kanban: urgência, score e antiguidade decidem a ordem de atendimento.*



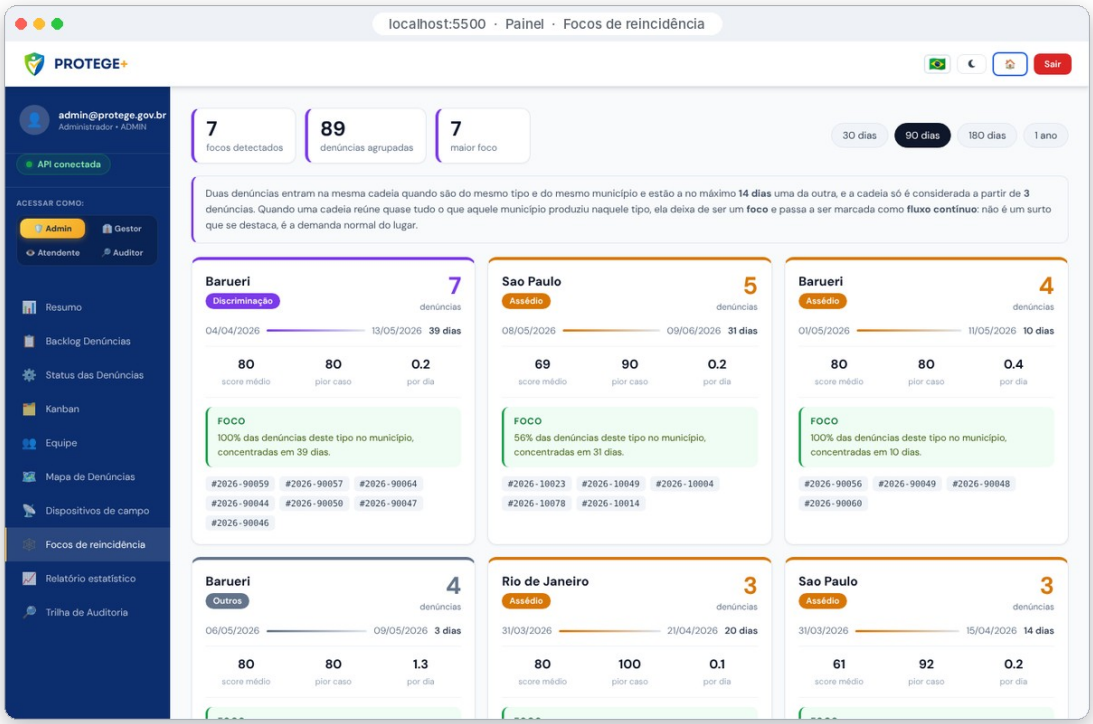
▲ **Figura 21.** *Relatório estatístico: correlação e regressão transformadas em conclusões com decisão sugerida.*



▲ **Figura 22.** *Trilha de auditoria: consultas sensíveis e recusas registradas, com a cadeia SHA-256 conferida.*



▲ **Figura 23.** *Dispositivos de campo: cada evento com a decisão do servidor.*



▲ **Figura 24.** *Focos de reincidência: cadeias de denúncias encontradas por grafo.*